National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

EE353: Computer Networks

Class: BSDS

CLO 1: Investigate and analyze the behavior of network traffic

Lab 6: Socket Programming (Client-Server Communication Using TCP and UDP in Python)

Date: 15-10-2026

Time: 02:00 PM to 05:00 PM

Lab Engineer: Mr. Hafiz Arslan Ramzan

Lab 6: Client-Server Communication Using TCP and UDP in Python

Objective of this Lab

The objective of this lab is to understand **client-server communication** and to implement it using **TCP and UDP socket programming in Python**. The lab starts with simple message exchange, moves on to **file sharing**, and finishes by building a small file-transfer service over both protocols and comparing their behavior using program output and Wireshark.

By the end of this lab, students will be able to:

- Explain the client-server model, sockets, IP addresses and port numbers.
- Create TCP and UDP servers and clients that exchange messages.
- Observe the TCP handshake and UDP datagrams in Wireshark.
- Explain why TCP is a byte stream and why one `recv()` call may return partial or merged data.
- Share a file between a client and a server over TCP, on one computer and (optionally) between two computers.
- Design a simple application protocol (request, header, data) with basic error handling.
- Transfer a file over UDP by splitting it into numbered chunks and reassembling it in order.
- Verify file integrity, measure transfer time and calculate throughput in KB/s.

Important Instructions

- Read the complete manual before starting. Understand each program before running it.
- Experiments and tasks are done individually; Experiment 6 (optional) is done in pairs.
- Keep all lab files in one folder (for example `Lab6`) and run every command from that folder.
- Always start the **server first**, then the client. Use a separate terminal window for each.
- Use only non-privileged ports (1024 and above). Only one program can use a port at a time.
- Do not copy code or answers from other students. You may use this manual and the official Python documentation.
- Include all required screenshots, answer the review questions and submit the report through the course LMS.

Tools/Software Requirements

- Python 3.11+ and any Python IDE (VS Code / PyCharm)
- Command Prompt / Terminal (two or three windows)
- Wireshark (on Windows, install Npcap with loopback support)
- For Experiment 6 only: a second computer on the same network (same Wi-Fi, mobile hotspot or LAN cable)

Lab Roadmap

Each experiment builds on the previous one. Approximate times are given as a guide.

Exp.	Title	What you will learn	Time
1	TCP Client-Server Messaging	Socket basics, connection-oriented communication	15 min
2	UDP Client-Server Messaging	Connectionless communication, datagrams	10 min
3	Observing Traffic in Wireshark	TCP handshake, UDP packets, ports, flags	20 min
4	TCP Is a Byte Stream	Why `recv()` may return partial or merged data	10 min
5	File Sharing Over TCP	Sending a file in chunks and saving it	15 min
6	File Sharing Between Two Computers (Optional)	Real IP addresses, firewall, network testing	20 min
7	Test File, Integrity Check and Timing	Creating a log file, SHA-256, throughput	10 min
8	Request-Response File Server Over TCP	Size header, exact-length receive, error handling	25 min
9	File Transfer Over UDP	Chunks, sequence numbers, duplicates, reassembly	25 min
10	Performance Comparison	Repeated runs, average time, throughput	20 min

1. Concepts You Need

1.1 Client-Server Model

In a **client-server** application one program, the **server**, waits for requests. Another program, the **client**, starts the communication, sends a request and receives a response.

Client → Request → Server **Client ← Response ← Server**

A web browser (client) requesting a page from a web server is the most common example. File transfer, e-mail and chat applications work the same way.

1.2 Sockets, IP Addresses and Ports

A **socket** is the endpoint an application uses to send and receive data over a network. It is identified by an **IP address** (which host), a **port number** (which application on that host) and a **transport protocol** (TCP or UDP). Python provides sockets through the built-in `socket` module (`import socket`).

- `127.0.0.1` is the **loopback** address: it always means "this computer". It lets you test a client and server on one machine.
- `0.0.0.0` (used when binding a server) means "listen on all network interfaces", so other computers can also connect.
- Ports 0-1023 are privileged. Use ports **1024 and above** for your own programs.

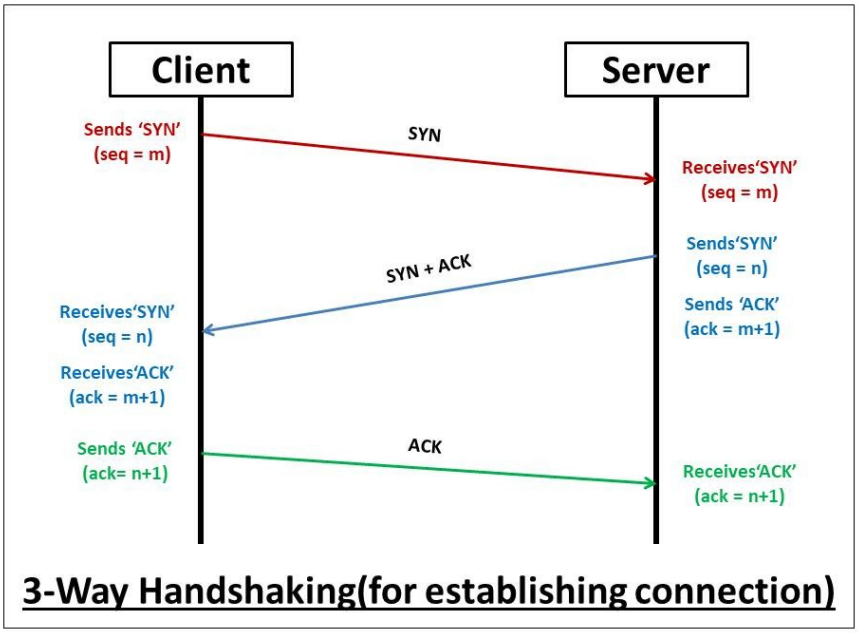
Ports used in this lab:

Port	Protocol	Used in
5000	TCP	Experiment 1 (messaging)
5001	UDP	Experiment 2 (messaging)
5004	TCP	Experiment 4 (byte-stream demo)
5002	TCP	Experiments 5-6 (simple file sharing)
6000	TCP	Experiment 8 (file server)

Port	Protocol	Used in
6001	UDP	Experiment 9 (file server)

1.3 TCP and UDP

TCP (Transmission Control Protocol) is connection-oriented. It sets up a connection with a **three-way handshake** (SYN, SYN-ACK, ACK) and then provides reliable, ordered delivery using acknowledgments, retransmission, flow control and congestion control. **UDP (User Datagram Protocol)** is connectionless: it sends independent datagrams with no handshake, no acknowledgments and no ordering guarantee, so it has lower overhead.



Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Delivery is guaranteed (retransmission)	No delivery guarantee
Ordering	Ordered byte stream	No ordering guarantee
Data unit	Stream of bytes (no message boundaries)	Datagrams (boundaries preserved)
Overhead	Higher	Lower
Typical use	Web, e-mail, file transfer	DNS, streaming, gaming, monitoring

1.4 Socket Programming Steps

Role	TCP	UDP
Server	`socket()` → `bind()` → `listen()` → `accept()` → `recv()` / `send()` → `close()`	`socket()` → `bind()` → `recvfrom()` / `sendto()` → `close()`
Client	`socket()` → `connect()` → `send()` / `recv()` → `close()`	`socket()` → `sendto()` / `recvfrom()` → `close()`

1.5 Important Socket Functions

Function	Purpose
`socket()`	Creates a socket (`SOCK_STREAM` = TCP, `SOCK_DGRAM` = UDP)
`bind()`	Assigns an IP address and port to a socket
`listen()` / `accept()`	Puts a TCP server in listening mode / accepts a connection and returns a new socket
`connect()`	Establishes a TCP connection to the server
`send()` / `sendall()`	Sends data; `sendall()` keeps sending until every byte has been handed to TCP
`recv()`	Receives **up to** the given number of bytes from a TCP socket
`sendto()` / `recvfrom()`	Sends / receives one UDP datagram (with the peer address)
`settimeout()`	Stops a blocking call from waiting forever
`close()`	Closes the socket

2. Experiments

Experiment 1: TCP Client-Server Messaging

Aim: Build a TCP server and client and exchange messages between them.

tcp_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(1)
print("TCP Server is waiting for a connection...")

conn, addr = server_socket.accept()
print("Connected by:", addr)

while True:
    data = conn.recv(1024)
    if not data:
        break
    message = data.decode()
    print("Client:", message)
    if message.lower() == "exit":
        break
    response = "Server received: " + message
    conn.send(response.encode())

conn.close()
server_socket.close()
print("Server closed.")
```

tcp_client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((HOST, PORT))
print("Connected to TCP server.")

while True:
    message = input("Enter message: ")
    client_socket.send(message.encode())
    if message.lower() == "exit":
        break
    response = client_socket.recv(1024)
    print("Server:", response.decode())

client_socket.close()
print("Client closed.")
```

How the code works

Code	Meaning
`socket.socket(AF_INET, SOCK_STREAM)`	`AF_INET` = IPv4, `SOCK_STREAM` = TCP
`bind((HOST, PORT))`	Attaches the server socket to 127.0.0.1, port 5000
`listen(1)`	Starts waiting for connection requests (queue of 1)
`accept()`	Blocks until a client connects. Returns `conn` (a new socket for this client) and `addr` (client IP, port)
`recv(1024)`	Receives up to 1024 bytes. An empty result (`b""`) means the other side closed the connection
`decode()` / `encode()`	Network data is bytes; these convert bytes ⇄ text
`connect((HOST, PORT))`	Client side: starts the three-way handshake with the server

Steps

1. Save the two programs in your `Lab6` folder.
2. **Terminal 1:** run `python tcp_server.py`. It prints `TCP Server is waiting for a connection...`
3. **Terminal 2:** run `python tcp_client.py`. It prints `Connected to TCP server.`
4. Type at least five different messages in the client, then type `exit`. Take a screenshot of both terminals.

Expected output (server, then client)

```
TCP Server is waiting for a connection...
Connected by: ('127.0.0.1', 52144)
Client: Hello Server

Enter message: Hello Server
Server: Server received: Hello Server
```

Experiment 2: UDP Client-Server Messaging

Aim: Build a UDP server and client and observe how it differs from TCP.

udp_server.py

```
import socket

HOST = "127.0.0.1"
PORT = 5001

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind((HOST, PORT))
print("UDP Server is waiting for messages...")

while True:
    data, addr = server_socket.recvfrom(1024)
    message = data.decode()
    print("Client:", message)
    if message.lower() == "exit":
        break
    response = "Server received: " + message
    server_socket.sendto(response.encode(), addr)

server_socket.close()
print("UDP Server closed.")
```

udp_client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5001

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
print("UDP Client started.")

while True:
    message = input("Enter message: ")
    client_socket.sendto(message.encode(), (HOST, PORT))
    if message.lower() == "exit":
        break
    data, server_address = client_socket.recvfrom(1024)
    print("Server:", data.decode())

client_socket.close()
print("Client closed.")
```

> **Key difference:** UDP has **no** `listen()`, `accept()` or `connect()`. The server simply calls `recvfrom()`, which returns the data **and** the sender's address; the client must give the destination address on every `sendto()`. Each `recvfrom()` returns exactly **one datagram**.

Steps

1. **Terminal 1:** run `python udp_server.py` (`UDP Server is waiting for messages...`).
2. **Terminal 2:** run `python udp_client.py`, send at least five messages, then type `exit`.
3. Take a screenshot of both terminals.

Experiment 3: Observing Traffic in Wireshark

Aim: See how the messages of Experiments 1 and 2 appear as packets on the network.

1. Start Wireshark **before** running the programs and select the **loopback** interface (Windows: *Npcap Loopback Adapter* / *Adapter for loopback traffic capture*; Linux: `lo`; macOS: `lo0`). Click Start capture.
2. Run Experiment 1 (TCP), send three messages and exit. Apply the display filter `tcp.port == 5000`.
3. Run Experiment 2 (UDP), send three messages and exit. Apply the display filter `udp.port == 5001`.
4. Click a packet and expand the layers in the details pane. Use *Right-click → Follow → TCP Stream / UDP Stream* to see the application data.

Look for the following and complete the comparison table in your report:

Observation	TCP (port 5000)	UDP (port 5001)
Packets before the first message	Handshake: SYN, SYN-ACK, ACK	None (data is sent immediately)
Packets for one message and its reply	Data packet (PSH, ACK) + ACK packets	One datagram each way
Fields you can see	Ports, sequence no., acknowledgment no., flags	Ports, length, checksum
End of communication	FIN / ACK packets	No closing exchange
Your own values	**Client port: 48110 Server port: 5000**	**Client port: 45601 Server port: 5001**

> **Note:** Wireshark shows *relative* sequence numbers (starting at 0). They count **bytes**, not packets. The client port is chosen automatically by the operating system (an ephemeral port).

Experiment 4: TCP Is a Byte Stream

Aim: Discover why a receiver cannot assume that one `send()` equals one `recv()`. This is essential for correct file transfer.

boundary_server.py (port 5004)

```
import socket
import time

HOST = "127.0.0.1"
PORT = 5004

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(1)
print("Boundary demo server waiting...")

conn, addr = server_socket.accept()
conn.sendall(b"AAAA")        # three separate send() calls
conn.sendall(b"BBBB")
conn.sendall(b"CCCC")
time.sleep(1)                # keep the connection open for a moment
conn.close()
server_socket.close()
```

boundary_client.py

```
import socket
import time

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5004))
time.sleep(0.5)              # let all three sends arrive first

data = client_socket.recv(1024)   # ONE recv() call
print("recv() returned:", data)
print("Number of bytes:", len(data))
client_socket.close()
```

1. Run `python boundary_server.py`, then `python boundary_client.py`.
2. The server made **three** `send` calls, but the client used **one** `recv()`. Observe the output.
3. Change `recv(1024)` to `recv(5)` and call it three times in the client. Observe how the data is split.

Expected output (step 2)

```
recv() returned: b'AAAABBBBCCCC'
Number of bytes: 12
```

> **Key point:** TCP keeps the **order** of bytes but not the **boundaries** of messages. Messages can merge, and one message can arrive in several pieces. Therefore a program that receives a file must (1) know **how many bytes** to expect and (2) keep calling `recv()` **in a loop** until all of them have arrived. UDP is different: every datagram arrives as a separate unit (if it arrives at all).

Experiment 5: File Sharing Over TCP

Aim: Make a server share a file and a client download it and save it locally (both on the same computer).

Prepare a file to share. Create `share_me.txt` in your `Lab3` folder using any editor, or run this one-line command:

```
python -c "open('share_me.txt','w').write('Hello from the server\n' * 500)"
```

share_server.py (port 5002)

```
import socket

HOST = "127.0.0.1"          # use "0.0.0.0" for two-computer sharing
PORT = 5002
FILE_NAME = "share_me.txt"

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST, PORT))
server_socket.listen(1)
print("File server waiting for a client...")

conn, addr = server_socket.accept()
print("Connected by:", addr)

sent = 0
with open(FILE_NAME, "rb") as f:
```

```
    while True:
        chunk = f.read(1024)
        if not chunk:                 # end of file reached
            break
        conn.sendall(chunk)
        sent += len(chunk)

print("File sent. Bytes sent:", sent)
conn.close()
server_socket.close()
```

share_client.py

```
import socket

SERVER_IP = "127.0.0.1"     # use the server computer's IP for two-computer sharing
PORT = 5002
SAVE_AS = "received_share_me.txt"

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((SERVER_IP, PORT))
print("Connected to file server.")

total = 0
with open(SAVE_AS, "wb") as f:
    while True:
        data = client_socket.recv(1024)
        if not data:                  # server closed the connection
            break
        f.write(data)
        total += len(data)

client_socket.close()
print("File saved as:", SAVE_AS)
print("Bytes received:", total)
```

How it works

- The file is opened in **binary mode** (`"rb"` / `"wb"`) so that any file type is copied byte for byte.
- The server reads the file in **1024-byte chunks** and sends each with `sendall()`.
- The client calls `recv()` in a loop and writes every piece to disk. `recv()` returning `b""` means the server closed the connection, so the file has ended.

Steps

1. Run `python share_server.py`, then (in another terminal) `python share_client.py`.
2. Compare the **sizes** of `share_me.txt` and `received_share_me.txt` (`dir` on Windows, `ls -l` on Linux/macOS). Both must be 11000 bytes.
3. Compare the **contents**: `fc /b share_me.txt received_share_me.txt` (Windows) or `cmp share_me.txt received_share_me.txt` (Linux/macOS). No difference should be reported.
4. Take screenshots of both terminals and of the comparison.

> **Limitation:** Here the client only knows the transfer is finished because the server **closed** the connection. If the connection broke half-way, the client could not tell a complete file from a partial one. Experiment 8 solves this by announcing the file size first.

Experiment 6 (Optional): File Sharing Between Two Computers

Aim: Run the server on one computer and the client on another. Work in pairs: Student A is the server, Student B is the client.

1. **Connect both computers to the same network** (same Wi-Fi, a mobile hotspot, or a LAN cable). Campus Wi-Fi sometimes blocks device-to-device traffic; if so, use a hotspot.
2. **Find the server's IP address.** On the server computer run `ipconfig` (Windows) or `ip a` / `ifconfig` (Linux/macOS) and note the IPv4 address, for example `192.168.1.25`.
3. **Test connectivity.** On the client computer run `ping 192.168.1.25` (use your partner's address). You must receive replies.
4. **Edit the server:** change `HOST = "127.0.0.1"` to `HOST = "0.0.0.0"` so that it accepts connections from other computers.
5. **Edit the client:** change `SERVER_IP = "127.0.0.1"` to the server's IP address.
6. **Allow the firewall.** When Windows asks, allow Python on **Private** networks.
7. **Run:** start `share_server.py` on computer A, then `share_client.py` on computer B.
8. **Verify:** on both computers run the command below and compare the two fingerprints. They must be identical.

Fingerprint command

```
python -c "import
hashlib;print(hashlib.sha256(open('share_me.txt','rb').read()).hexdigest())"
(on the client use received_share_me.txt)
```

1. **Swap roles** and repeat.
2. **Wireshark:** capture on the Wi-Fi/Ethernet interface with filter `tcp.port == 5002`. Notice that the source and destination IP addresses are now **different**, unlike loopback traffic.

If something goes wrong, check this table:

Symptom	Likely cause	Fix
Connection refused	Server not running, or wrong IP/port	Start the server first; recheck IP and port
Connection timed out	Firewall or network blocks traffic; different networks	Allow Python in firewall; use the same hotspot/LAN
Address already in use	Old server still running on that port	Close it, wait a moment, or change the port
FileNotFoundError	File is not in the current folder	Run from the folder that contains the file

Experiment 7: Test File, Integrity Check and Timing

Aim: Create a realistic text log file, learn how to prove that two files are identical, and learn how to measure time and throughput.

Concepts

- **Integrity check:** a SHA-256 hash is a fingerprint of a file. If even one byte differs, the fingerprint changes completely. Comparing hashes is more reliable than comparing sizes.
- **Timing:** `time.perf_counter()` returns a high-resolution clock. Elapsed time = end − start.

- **Throughput:** application-level throughput (KB/s) = file size in KB ÷ elapsed time in seconds, where file size in KB = bytes ÷ 1024.

generate_log.py

```
import random
from datetime import datetime, timedelta

FILE_NAME = "sensor_log.txt"
TARGET_KB = 300                      # approximate file size
LOCATIONS = ["Library", "CS-Block", "Cafeteria", "Hostel-A", "Admin-Block"]

reading_time = datetime(2026, 9, 23, 8, 0, 0)
size = 0

with open(FILE_NAME, "w", newline="\n") as f:
    header = "timestamp,location,temperature_C,humidity_pct,air_quality_AQI\n"
    f.write(header)
    size += len(header)

    while size < TARGET_KB * 1024:
        line = "{},{},{:.1f},{:.1f},{}\n".format(
            reading_time.strftime("%Y-%m-%d %H:%M:%S"),
            random.choice(LOCATIONS),
            random.uniform(18.0, 38.0),      # temperature
            random.uniform(25.0, 80.0),      # humidity
            random.randint(30, 180))         # air quality index
        f.write(line)
        size += len(line)
        reading_time += timedelta(seconds=5)

print("Created", FILE_NAME, "with", size, "bytes (", round(size / 1024, 1), "KB )")
```

file_tools.py

```
import hashlib
import os
import time

def file_hash(path):
    """Return the SHA-256 fingerprint of a file."""
    sha = hashlib.sha256()
    with open(path, "rb") as f:
        while True:
            block = f.read(65536)
            if not block:
                break
            sha.update(block)
    return sha.hexdigest()

if __name__ == "__main__":
    name = "sensor_log.txt"
    size_bytes = os.path.getsize(name)

    start = time.perf_counter()
    digest = file_hash(name)
    elapsed = time.perf_counter() - start

    print("File       :", name)
    print("Size       :", size_bytes, "bytes")
    print("SHA-256    :", digest)
    print("Hash time  : {:.6f} s".format(elapsed))
    print("Throughput : {:.2f} KB/s".format((size_bytes / 1024) / elapsed))
```

Steps

1. Run `python generate_log.py`. A file `sensor_log.txt` of about 300 KB is created. Open it and look at the first lines.
2. Create a folder named `server_files` and **copy** `sensor_log.txt` into it. The file servers in Experiments 8 and 9 share files from this folder.
3. Run `python file_tools.py` and note the size, SHA-256 value and hash time.
4. Edit `TARGET_KB` to create other sizes (for example 100 and 1000) and note that the file size can range from 100 KB to 1 MB.

Sample lines of sensor_log.txt

```
timestamp,location,temperature_C,humidity_pct,air_quality_AQI
2026-09-23 08:00:00,Library,27.4,51.3,112
2026-09-23 08:00:05,Cafeteria,31.0,44.8,76
2026-09-23 08:00:10,CS-Block,22.6,63.2,141
```

Experiment 8: Request-Response File Server Over TCP

Aim: Build a TCP file server that a client can ask for a file by name. The server announces the file size, the client receives exactly that many bytes, saves the file and verifies it. Common errors must be handled.

Protocol design

A **protocol** is an agreement about what each side sends and in what order. Ours is text for the header and raw bytes for the file:

Step	Direction	Message
1	Client → Server	`filename` followed by a newline character
2	Server → Client	`OK <size> <sha256>` + newline, **or** `ERROR <reason>` + newline
3	Server → Client	Exactly `<size>` bytes of file data (sent in 1024-byte chunks)
4	Client	Receives until `<size>` bytes are counted, saves the file, compares the hash

Key ideas used in the code

- **Size header:** the client knows exactly when the file ends, without relying on the connection closing.
- **Exact-length receive loop:** `while received < size:` with `recv(min(BUFFER_SIZE, size - received))` handles partial `recv()` results and never reads past the file.
- **Premature closure:** if `recv()` returns `b""` before all bytes arrive, the client reports `Connection closed early`.
- **Filename validation:** names containing folders or `..` (for example `../secret.txt`) are rejected so clients cannot read files outside `server_files`.
- **Robust server:** it handles one client after another in a `while True` loop, and a `try/except` ensures one failed transfer does not stop the server.

tcp_file_server.py (port 6000)

```
import hashlib
import os
import socket
import time
```

```
HOST = "127.0.0.1"          # "0.0.0.0" = accept other computers too
PORT = 6000
FILES_FOLDER = "server_files"
CHUNK_SIZE = 1024
SLOW_DEMO = False           # True = pause after each chunk (for testing)


def recv_line(conn, limit=255):
    """Read bytes one at a time until a newline is found."""
    line = b""
    while not line.endswith(b"\n"):
        ch = conn.recv(1)
        if not ch:
            raise ConnectionError("Client closed the connection early")
        line += ch
        if len(line) > limit:
            raise ValueError("Request line too long")
    return line.decode().strip()


def is_valid_name(name):
    """Accept plain file names only (no folders, no '..')."""
    return name != "" and name == os.path.basename(name) and ".." not in name


def sha256_of(path):
    sha = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(65536), b""):
            sha.update(block)
    return sha.hexdigest()


def handle_client(conn, addr):
    filename = recv_line(conn)
    print("Request from", addr, "for:", repr(filename))

    if not is_valid_name(filename):
        conn.sendall(b"ERROR Invalid filename\n")
        print("Rejected: invalid filename")
        return

    path = os.path.join(FILES_FOLDER, filename)
    if not os.path.isfile(path):
        conn.sendall(b"ERROR File not found\n")
        print("Rejected: file not found")
        return

    size = os.path.getsize(path)
    header = "OK {} {}\n".format(size, sha256_of(path))
    conn.sendall(header.encode())

    start = time.perf_counter()
    sent = 0
    with open(path, "rb") as f:
        while True:
            chunk = f.read(CHUNK_SIZE)
            if not chunk:
                break
            conn.sendall(chunk)
            sent += len(chunk)
            if SLOW_DEMO:
                time.sleep(0.05)
    elapsed = time.perf_counter() - start
    print("Sent {} bytes in {:.4f} s".format(sent, elapsed))
```

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(5)
print("TCP file server listening on {}:{}".format(HOST, PORT))

while True:
    conn, addr = server_socket.accept()
    try:
        handle_client(conn, addr)
    except Exception as error:
        print("Transfer failed:", error)
    finally:
        conn.close()
```

tcp_file_client.py

```
import hashlib
import socket
import sys
import time

SERVER_IP = sys.argv[2] if len(sys.argv) > 2 else "127.0.0.1"
PORT = 6000
BUFFER_SIZE = 4096


def recv_line(sock, limit=255):
    line = b""
    while not line.endswith(b"\n"):
        ch = sock.recv(1)
        if not ch:
            raise ConnectionError("Server closed before replying")
        line += ch
        if len(line) > limit:
            raise ValueError("Reply line too long")
    return line.decode().strip()


filename = sys.argv[1] if len(sys.argv) > 1 else input("File to request: ")
save_as = "received_tcp_" + filename.replace("/", "_").replace("\\", "_")

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.settimeout(10)

try:
    client.connect((SERVER_IP, PORT))
    start = time.perf_counter()
    client.sendall((filename + "\n").encode())

    reply = recv_line(client).split()
    if reply[0] != "OK":
        print("Server error:", " ".join(reply[1:]))
        sys.exit(1)

    size, expected_hash = int(reply[1]), reply[2]
    received = 0
    sha = hashlib.sha256()

    with open(save_as, "wb") as f:
        while received < size:
            data = client.recv(min(BUFFER_SIZE, size - received))
            if not data:
```

```
                raise ConnectionError("Connection closed early: got {} of {} bytes"
                                      .format(received, size))
            f.write(data)
            sha.update(data)
            received += len(data)

    elapsed = time.perf_counter() - start
    throughput = (received / 1024) / elapsed
    ok = sha.hexdigest() == expected_hash

    print("Protocol       : TCP")
    print("Filename       :", filename)
    print("Bytes received :", received)
    print("Elapsed time   : {:.4f} s".format(elapsed))
    print("Throughput     : {:.2f} KB/s".format(throughput))
    print("File integrity :", "MATCH" if ok else "MISMATCH")

except (ConnectionError, socket.timeout, OSError) as error:
    print("Transfer failed:", error)
finally:
    client.close()
```

Steps

1. Make sure `server_files/sensor_log.txt` exists. Run `python tcp_file_server.py`.
2. In another terminal run `python tcp_file_client.py sensor_log.txt`. The file is saved as `received_tcp_sensor_log.txt`.
3. **Error test 1:** run `python tcp_file_client.py missing.txt`.
4. **Error test 2:** run `python tcp_file_client.py ../secret.txt`.
5. **Error test 3 (server not running):** stop the server and run the client again.
6. **Error test 4 (premature closure):** set `SLOW_DEMO = True` in the server, restart it and start a transfer. While it is running, press **Ctrl+C** in the server terminal. Set `SLOW_DEMO` back to `False` afterwards.

Expected output (values will differ on your computer)

```
Protocol       : TCP
Filename       : sensor_log.txt
Bytes received : 307226
Elapsed time   : 0.0024 s
Throughput     : 124493.26 KB/s
File integrity : MATCH
```

Test	Expected client message
Missing file	`Server error: File not found`
Invalid filename	`Server error: Invalid filename`
Server not running	`Transfer failed: ... Connection refused`
Server stopped during transfer	`Transfer failed: Connection closed early: got X of Y bytes`

Optional: change `CHUNK_SIZE` to 4096 and 65536 and observe how throughput changes.

Experiment 9: File Transfer Over UDP

Aim: Transfer the same file using UDP. Because UDP gives no ordering or delivery guarantee, the application must add its own **sequence numbers**, **chunk count**, **duplicate detection** and **reassembly**.

Packet design

Datagram	Content
Request (client → server)	Text: `GET <filename>`
Reply (server → client)	Text: `OK <size> <total_chunks> <sha256>` or `ERROR <reason>`
Data (server → client)	4-byte **sequence number** + up to 1024 bytes of file data

- `struct.pack("!I", seq)` converts the sequence number into 4 bytes (`!` = network byte order, `I` = unsigned 32-bit integer). `struct.unpack` reverses it.
- Total chunks = file size ÷ 1024, rounded **up**. A 307226-byte file needs 301 chunks.
- The client stores chunks in a dictionary keyed by sequence number. A number that is already stored is a **duplicate**; a number outside `0 … total−1` is **unexpected**. Sorting by key restores the correct **order**.
- The client stops when all chunks have arrived. If nothing arrives for 2 seconds it reports the **missing** chunk numbers.
- The receive buffer is enlarged (`SO_RCVBUF`) and the server pauses 1 ms every 50 packets. Without this, a fast sender can overflow the receiver's buffer and the operating system silently drops datagrams.

udp_file_server.py (port 6001)

```
import hashlib
import os
import socket
import struct
import time

HOST = "127.0.0.1"          # "0.0.0.0" = accept other computers too
PORT = 6001                 # different from the TCP file server (6000)
FILES_FOLDER = "server_files"
CHUNK_SIZE = 1024           # payload bytes per datagram
PACE_EVERY = 50             # short pause after this many packets

TEST_DUP_REORDER = False    # True = duplicate one chunk, swap two chunks
TEST_DROP_CHUNK = None      # e.g. 7 = never send chunk 7 (simulates packet loss)


def is_valid_name(name):
    return name != "" and name == os.path.basename(name) and ".." not in name


def sha256_of(path):
    sha = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(65536), b""):
            sha.update(block)
    return sha.hexdigest()


server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))
print("UDP file server listening on {}:{}".format(HOST, PORT))
```

```
while True:
    request, addr = server.recvfrom(1024)
    text = request.decode(errors="replace").strip()
    parts = text.split(" ", 1)
    if len(parts) != 2 or parts[0] != "GET":
        server.sendto(b"ERROR Bad request", addr)
        continue

    filename = parts[1]
    print("Request from", addr, "for:", repr(filename))

    if not is_valid_name(filename):
        server.sendto(b"ERROR Invalid filename", addr)
        continue
    path = os.path.join(FILES_FOLDER, filename)
    if not os.path.isfile(path):
        server.sendto(b"ERROR File not found", addr)
        continue

    size = os.path.getsize(path)
    total_chunks = (size + CHUNK_SIZE - 1) // CHUNK_SIZE
    info = "OK {} {} {}".format(size, total_chunks, sha256_of(path))
    server.sendto(info.encode(), addr)

    with open(path, "rb") as f:
        data = f.read()

    order = list(range(total_chunks))
    if TEST_DUP_REORDER and total_chunks > 12:
        order[10], order[11] = order[11], order[10]   # out-of-order
        order.insert(4, 3)                            # chunk 3 sent twice
    if TEST_DROP_CHUNK is not None and TEST_DROP_CHUNK in order:
        order.remove(TEST_DROP_CHUNK)                 # lost chunk

    sent = 0
    for count, seq in enumerate(order, start=1):
        payload = data[seq * CHUNK_SIZE:(seq + 1) * CHUNK_SIZE]
        server.sendto(struct.pack("!I", seq) + payload, addr)
        sent += 1
        if count % PACE_EVERY == 0:
            time.sleep(0.001)
    print("Sent {} datagrams ({} chunks in the file)".format(sent, total_chunks))
```

udp_file_client.py

```
import hashlib
import socket
import struct
import sys
import time

SERVER_IP = sys.argv[2] if len(sys.argv) > 2 else "127.0.0.1"
PORT = 6001
TIMEOUT = 2.0              # seconds to wait for the next datagram
MAX_TRIES = 3              # request attempts
HEADER = struct.calcsize("!I")

filename = sys.argv[1] if len(sys.argv) > 1 else input("File to request: ")
save_as = "received_udp_" + filename.replace("/", "_").replace("\\", "_")

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 4 * 1024 * 1024)
client.settimeout(TIMEOUT)

# ---- Step 1: send the request and wait for the OK / ERROR reply ----
```

```
start = time.perf_counter()
reply = None
for attempt in range(1, MAX_TRIES + 1):
    client.sendto(("GET " + filename).encode(), (SERVER_IP, PORT))
    try:
        reply, _ = client.recvfrom(2048)
        break
    except socket.timeout:
        print("No reply (attempt {} of {})".format(attempt, MAX_TRIES))

if reply is None:
    print("Transfer failed: server did not respond")
    sys.exit(1)

fields = reply.decode(errors="replace").split()
if fields[0] != "OK":
    print("Server error:", " ".join(fields[1:]))
    sys.exit(1)

size, total_chunks, expected_hash = int(fields[1]), int(fields[2]), fields[3]

# ---- Step 2: receive chunks until all have arrived ----
chunks = {}
duplicates = 0
unexpected = 0

try:
    while len(chunks) < total_chunks:
        packet, _ = client.recvfrom(HEADER + 65535)
        if len(packet) < HEADER:
            unexpected += 1
            continue
        seq = struct.unpack("!I", packet[:HEADER])[0]
        if seq >= total_chunks:
            unexpected += 1               # sequence number outside the valid range
        elif seq in chunks:
            duplicates += 1               # already received
        else:
            chunks[seq] = packet[HEADER:]
except socket.timeout:
    pass

elapsed = time.perf_counter() - start
client.close()

# ---- Step 3: check and reassemble ----
missing = [n for n in range(total_chunks) if n not in chunks]
if missing:
    print("Transfer incomplete. Missing chunks:", missing[:20])
    print("Received {} of {} chunks".format(len(chunks), total_chunks))
    sys.exit(1)

data = b"".join(chunks[n] for n in range(total_chunks))
with open(save_as, "wb") as f:
    f.write(data)

throughput = (len(data) / 1024) / elapsed
ok = hashlib.sha256(data).hexdigest() == expected_hash

print("Protocol          : UDP")
print("Filename          :", filename)
print("Bytes received    :", len(data), "of", size)
print("Chunks            :", total_chunks)
print("Duplicates ignored:", duplicates)
print("Unexpected packets:", unexpected)
print("Elapsed time      : {:.4f} s".format(elapsed))
```

```
print("Throughput        : {:.2f} KB/s".format(throughput))
print("File integrity    :", "MATCH" if ok else "MISMATCH")
```

Steps

1. Run `python udp_file_server.py`, then `python udp_file_client.py sensor_log.txt`.
2. **Duplicate/out-of-order test:** set `TEST_DUP_REORDER = True` in the server and restart it. Run the client. It should report `Duplicates ignored: 1` and still show `MATCH`.
3. **Packet-loss test:** set `TEST_DUP_REORDER = False` and `TEST_DROP_CHUNK = 7`, restart the server and run the client. The client reports `Missing chunks: [7]` after the 2-second timeout. Reset to `None` afterwards.
4. **Error tests:** request `missing.txt` and `../secret.txt`; then stop the server and run the client (it tries three times).

Expected output (values will differ on your computer)

```
Protocol          : UDP
Filename          : sensor_log.txt
Bytes received    : 307226 of 307226
Chunks            : 301
Duplicates ignored: 0
Unexpected packets: 0
Elapsed time      : 0.0095 s
Throughput        : 31681.85 KB/s
File integrity    : MATCH
```

> **Think about it:** In the loss test the file was **not** completed. TCP repairs loss automatically; with UDP the application must do it. *Optional challenge:* make the client send a second request listing the missing chunk numbers so that the server resends only those.

Experiment 10: Performance Comparison of TCP and UDP

Aim: Measure and compare the transfer time and throughput of both implementations using the same file under the same conditions.

Method

1. Start both servers (`tcp_file_server.py` and `udp_file_server.py`) with all test settings switched **off** (`SLOW_DEMO = False`, `TEST_DUP_REORDER = False`, `TEST_DROP_CHUNK = None`). Close Wireshark and heavy programs.
2. Run `python tcp_file_client.py sensor_log.txt` **three times** and record each elapsed time.
3. Run `python udp_file_client.py sensor_log.txt` **three times** and record each elapsed time.
4. Calculate the average time and throughput for each protocol and fill the table.

Formulas: **Average time = (T1 + T2 + T3) ÷ 3** **Throughput (KB/s) = (file size in bytes ÷ 1024) ÷ average time (s)**

Calculation helper

```
times = [0.0024, 0.0022, 0.0021]      # your three elapsed times (seconds)
size_bytes = 307226

average = sum(times) / len(times)
throughput = (size_bytes / 1024) / average
print("Average time : {:.4f} s".format(average))
print("Throughput   : {:.2f} KB/s".format(throughput))
```

Protocol	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Throughput (KB/s)
TCP	0.0022	0.0032	0.0014	0.0023	132363
UDP	0.0092	0.0089	0.0091	0.0091	33091

Wireshark comparison (optional)

Capture one TCP and one UDP transfer (filters `tcp.port == 6000` and `udp.port == 6001`). Open *Statistics → Conversations* and note the number of packets and bytes for each. TCP shows acknowledgment packets that UDP does not. On loopback, TCP may use very large segments, so packet counts differ from a real network.

> **Interpreting results:** On one computer there is no real packet loss or congestion, and the UDP program includes a small pacing delay, so the numbers reflect **our implementations** as well as the protocols. Do not conclude that one protocol is always faster. Discuss: what does TCP provide automatically that the UDP program had to build by hand?

3. Common Errors and Fixes

Message / Problem	Meaning	Fix
`ConnectionRefusedError`	No server is listening on that IP/port	Start the server first; check IP and port
`OSError: Address already in use`	The port is still held by another program	Close the old server or change the port
Client seems frozen	Waiting for data that never comes (`recv()` blocks)	Check the server is running; use `settimeout()`
`FileNotFoundError`	File not in the working folder	Run from the correct folder; check the file name
Wireshark shows no packets	Wrong interface selected	Select the loopback interface for 127.0.0.1 traffic
Hash shows `MISMATCH`	File corrupted or incomplete	Repeat the transfer; check the receive loop

4. Lab Tasks

Complete the following tasks. Provide code and screenshots for each.

Group A: Messaging (Experiments 1 and 2)

1. **Welcome message:** modify the TCP server so that it sends `Welcome to the Computer Networks Lab` as soon as a client connects.

2. **Message counter:** modify the TCP server to number the messages it receives (`Message 1: Hello`, `Message 2: ...`).
3. **Reverse server:** modify the TCP server to reply with the reversed text (`Computer Networks` → `skrowteN retupmoC`). Test three messages.
4. **Uppercase server:** modify the UDP server to reply with the message in uppercase.
5. **TCP calculator:** the client sends two numbers and an operator (`+ - * /`); the server returns the result. Test three operations.
6. **UDP echo:** make the UDP server return exactly the message it received.

Group B: Wireshark (Experiment 3)

7. **Packet details:** capture TCP and UDP communication. Record source/destination IP, source/destination ports, TCP flags in the handshake, the packet carrying the first message, and the UDP length and checksum. Paste screenshots.
8. **TCP vs UDP:** send the same message with both protocols and compare packets exchanged, connection set-up, headers and application data.

Group C: File sharing and transfer (Experiments 5-10)

9. **TCP file transfer:** transfer `sensor_log.txt` and a second text file of a different size using Experiment 8. Show `MATCH` for both.
10. **Error handling:** show screenshots of the missing-file, invalid-filename and premature-closure tests.
11. **UDP file transfer:** transfer the file using Experiment 9 and show the duplicate/out-of-order test and the packet-loss test.
12. **Performance table:** present the completed table of Experiment 10 with three runs per protocol, averages and throughput, followed by a short comparison paragraph (5-6 lines).
13. **(Optional) Two computers:** complete Experiment 6 and include screenshots from both computers and a Wireshark capture showing two different IP addresses.

5. Review Questions

1. What is the difference between connection-oriented and connectionless communication? Give one example of each.
2. Why does a server call `bind()` while a client usually does not?
3. What do `listen()` and `accept()` do? What does `accept()` return?
4. Why can one `recv()` return fewer bytes than were sent, or several messages joined together? How does the TCP file client handle this?
5. Why does the TCP file server send the file size before the file data?
6. What is the difference between `127.0.0.1` and `0.0.0.0`?
7. In the UDP file transfer, why is a sequence number needed? What can happen to UDP datagrams on the way?
8. Why is comparing SHA-256 hashes better than comparing file sizes?
9. How is throughput calculated? Why do we average several runs?
10. Why must a file server reject filenames such as `../secret.txt`?

6. Submission Requirements

Submit a lab report containing:

1. Answers to all review questions.
2. Screenshots of TCP and UDP client/server execution (Experiments 1, 2, 5, 8, 9).
3. Relevant Wireshark screenshots and the completed comparison table.
4. Modified code and screenshots for all lab tasks, with brief explanations where required.
5. The completed performance table of Experiment 10 and your comparison paragraph.

All screenshots must clearly show the relevant output or packet information.

Conclusion

In this lab we built client-server applications using Python sockets. We exchanged messages over TCP and UDP, observed the resulting packets in Wireshark, and learned that TCP is a reliable byte stream while UDP delivers independent datagrams. We then shared files over TCP, designed a request-response protocol with a size header and error handling, and implemented a UDP transfer using sequence numbers and reassembly. Finally, we verified file integrity and compared the transfer time and throughput of both protocols.

Lab 6 Report: Socket Programming

Arqam Waheed · 553514 · BSDS 3A · submitted 15 October 2026

Everything below was produced by running the programs on Ubuntu with Python 3.10.12 and capturing the traffic with Wireshark 3.6.2 on the loopback interface **lo**. The blanks inside the manual have also been filled in: the port table in Experiment 3 and the performance table in Experiment 10.

My code is organised differently from the listings in the manual. Each experiment is one file that holds both roles and is selected on the command line (for example **python3 messaging.py server** and **python3 messaging.py client**), so the protocol is defined once and the two sides cannot drift apart. The wire format is exactly the one the manual specifies, so these programs interoperate with the manual's versions.

File	Covers	How it is run
messaging.py	Experiment 1, Group A tasks 1-3	messaging.py server | client "msg" …
udp_messaging.py	Experiment 2, Group A tasks 4 and 6	udp_messaging.py server --mode upper | echo
calculator.py	Group A task 5	calculator.py server | client "12 + 8"
stream_boundaries.py	Experiment 4	stream_boundaries.py [--split]
tcp_share.py	Experiments 5 and 6	tcp_share.py server [--lan] | client [--host IP]
make_log.py / integrity.py	Experiment 7	make_log.py 300 sensor_log.txt
tcp_file_server.py / tcp_file_client.py	Experiment 8	tcp_file_client.py sensor_log.txt
udp_file_server.py / udp_file_client.py	Experiment 9 + optional challenge	udp_file_server.py [--dup] [--drop 7]
benchmark.py	Experiment 10	benchmark.py sensor_log.txt

Experiments 1 and 2: messaging over TCP and UDP

The TCP server wraps the accepted socket in a file object with makefile(), so one line of the conversation is one iteration of the loop and the three Group A changes (welcome, counter, reverse) are all visible in it.

```
        with conn, conn.makefile("rwb", buffering=0) as stream:
            stream.write((WELCOME + "\n").encode())          # task 1

            counter = 0
            for raw in stream:                               # one iteration per line
                text = raw.decode().rstrip("\n")
                counter += 1                                 # task 2
                print(f"[server] message {counter}: {text}")
                if text.lower() == "exit":
                    break
                stream.write((transform(counter, text) + "\n").encode())

def transform(number: int, text: str) -> str:
    return f"[{number}] {text[::-1]}"                        # task 3
```

Figure 1: messaging.py: the server loop

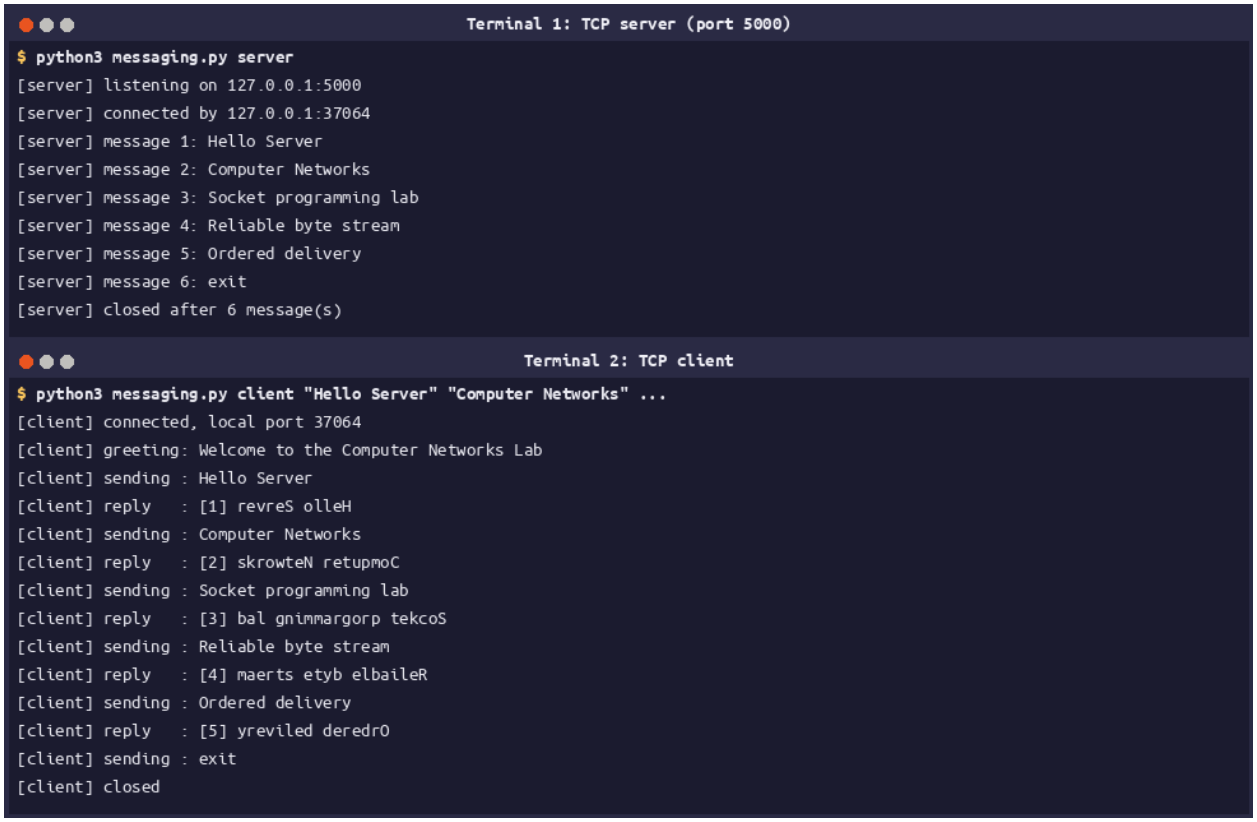


Figure 2: Five messages, numbered and answered in reverse, then exit

The client was accepted from port 37064 and the greeting arrived before the first message was typed.

The UDP version keeps the same shape but the reply rule is a dictionary entry, so Group A task 4 (uppercase) and task 6 (echo) are the same program started with a different --mode.

```
MODES = {
    "upper": lambda data: f"Server received: {data.decode().upper()}".encode(),
    "echo":  lambda data: data,                        # exactly what arrived
}


        data, peer = server.recvfrom(1024)             # one datagram per call
        ...
        server.sendto(reply_for(data), peer)
```

Figure 3: udp_messaging.py

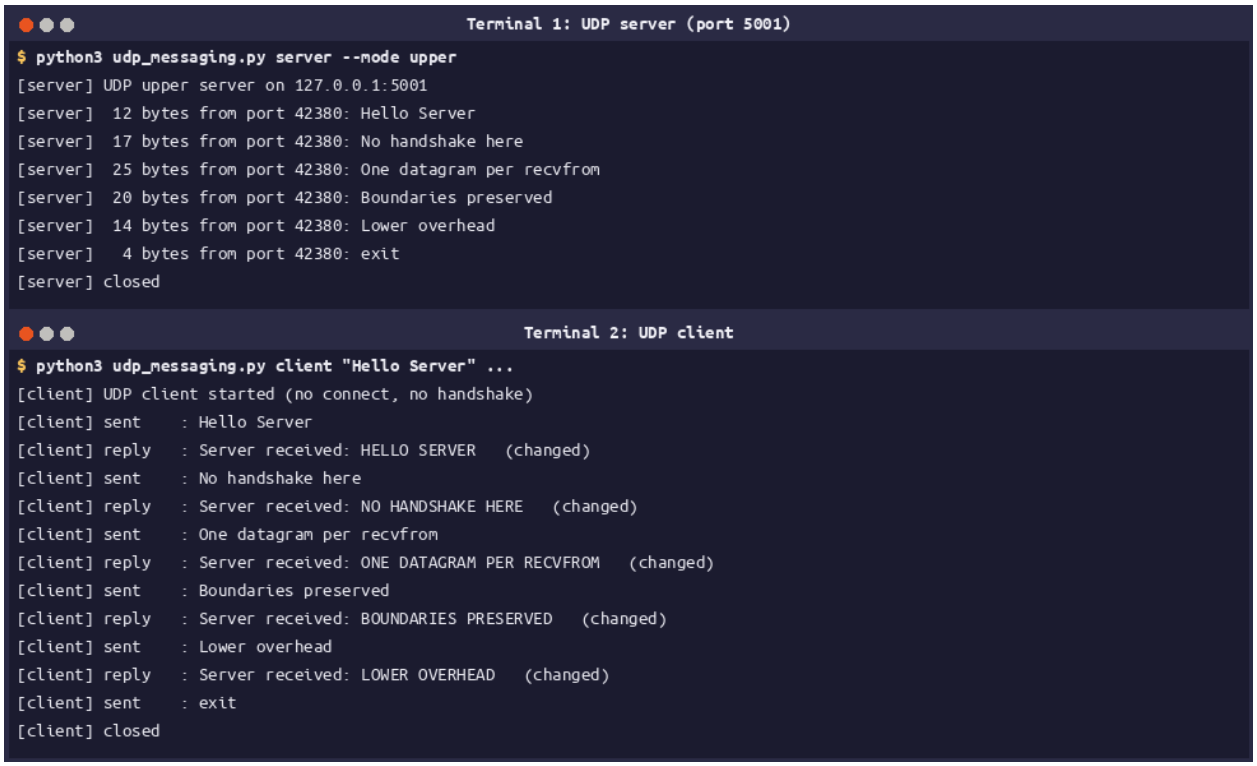


Figure 4: The same five messages over UDP, answered in uppercase (client port 45601)

Experiment 3: the traffic in Wireshark

Both conversations are in lab6b_msg.pcapng (46 packets). **tcp.port == 5000** matches 20 of them and **udp.port == 5001** matches 16, of which the last five are the echo test of task 6. The remaining 10 packets are DNS lookups from other programs, which the loopback interface carried during the capture.

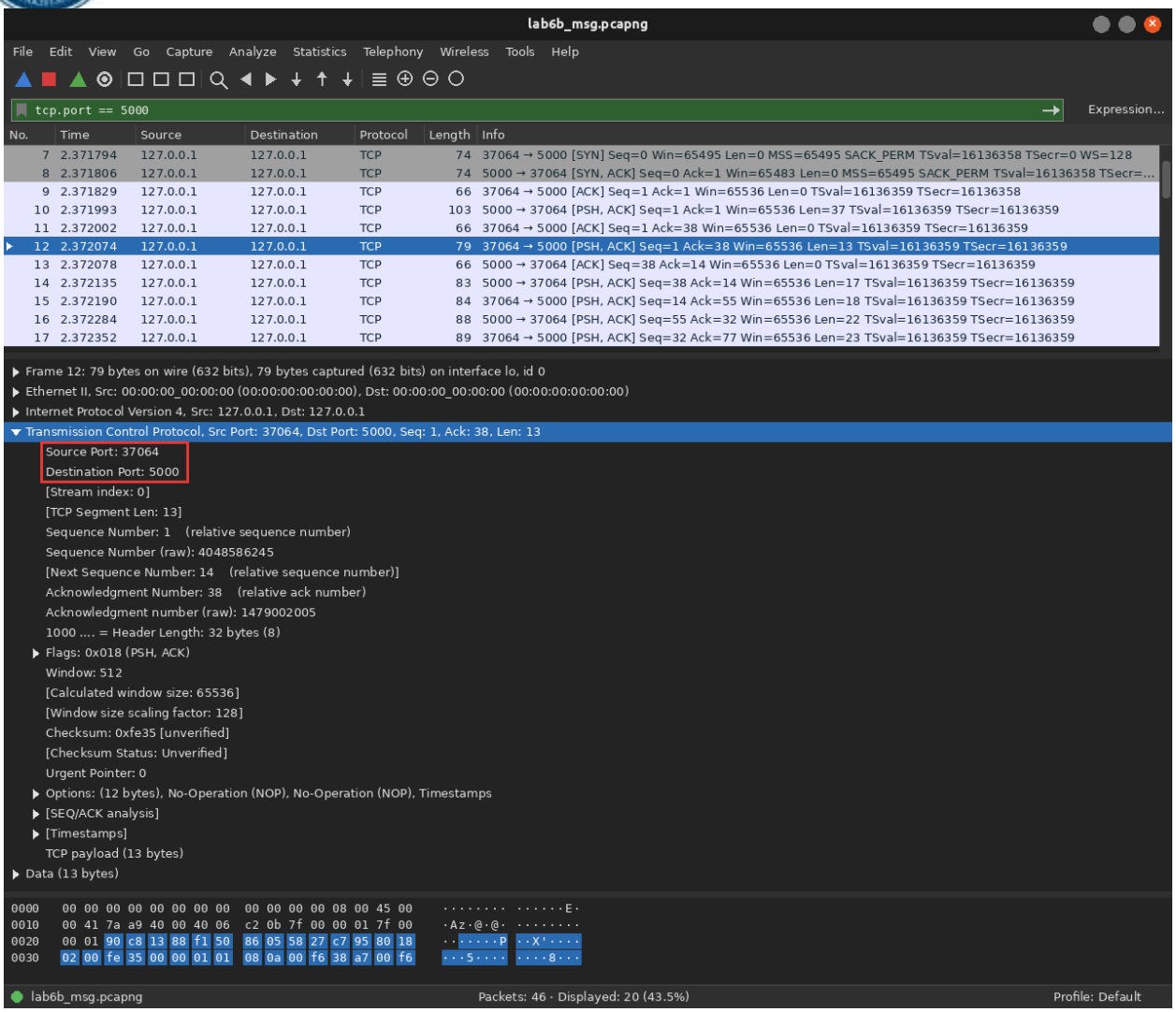


Figure 5: Filter tcp.port == 5000, frame 12 selected: the client's first message (13 bytes)

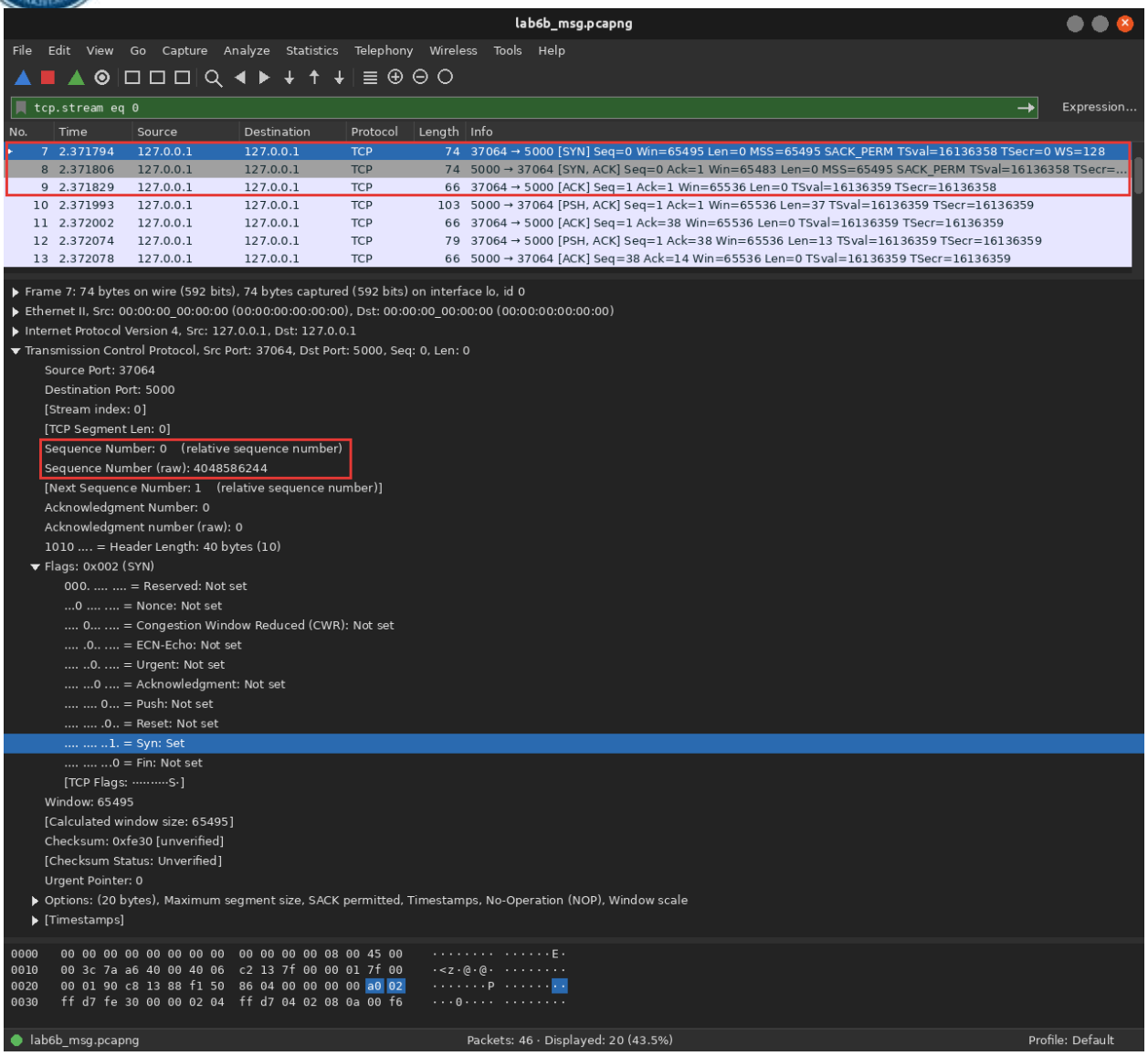


Figure 6: Frames 7-9: SYN, SYN/ACK, ACK. No data can move until all three are done

Packet	Frame	Ports	Flags	Seq / Ack
SYN	7	37064 → 5000	0x002 (SYN)	Seq 0
SYN/ACK	8	5000 → 37064	0x012 (SYN, ACK)	Seq 0, Ack 1
ACK	9	37064 → 5000	0x010 (ACK)	Seq 1, Ack 1

The SYN asks for a connection and announces the client's initial sequence number; the SYN/ACK acknowledges it with ISN+1 and carries the server's own; the final ACK confirms the server's number so both directions are open. The initial round trip was 0.000035 s.

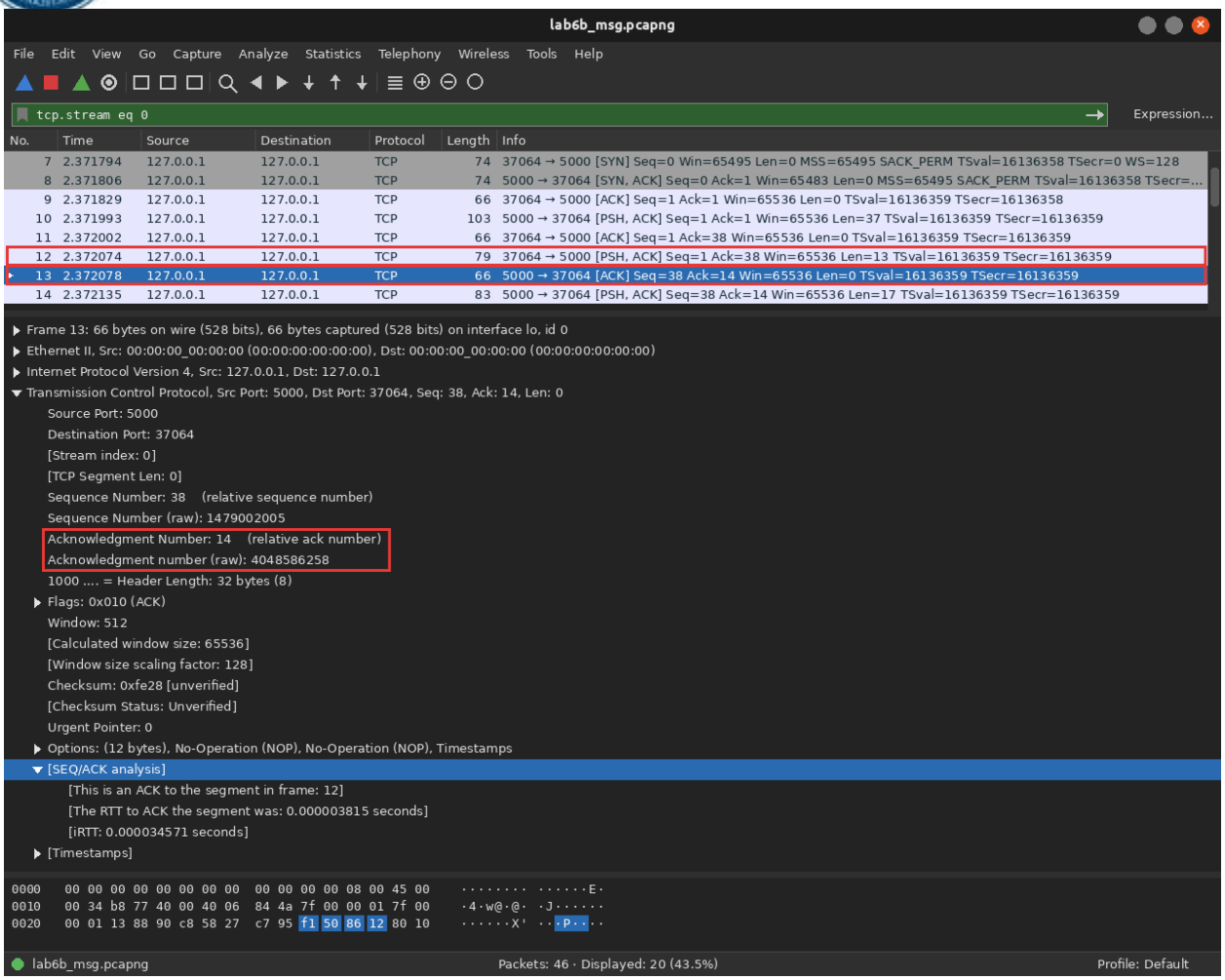


Figure 7: Frame 13 acknowledges the 13 data bytes of frame 12 with Ack = 14

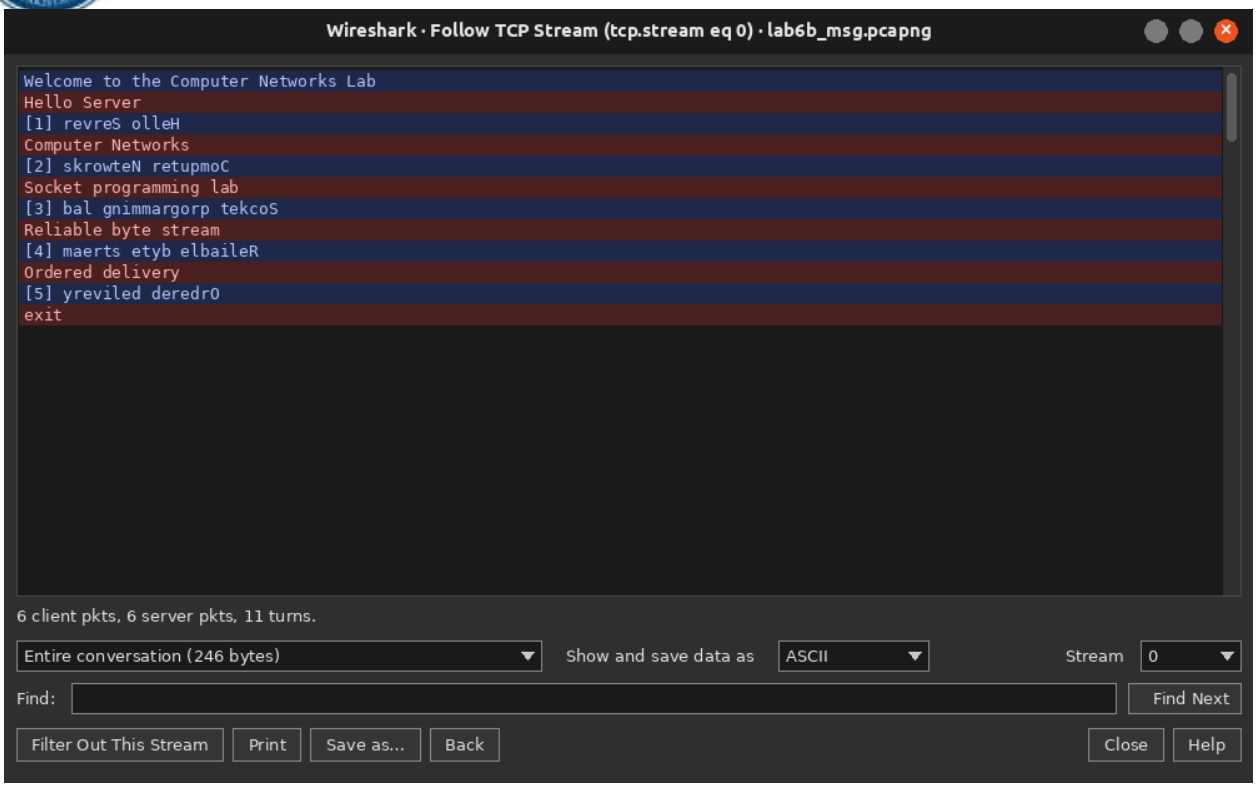


Figure 8: Follow TCP Stream: 6 client packets, 6 server packets, 11 turns, 246 bytes

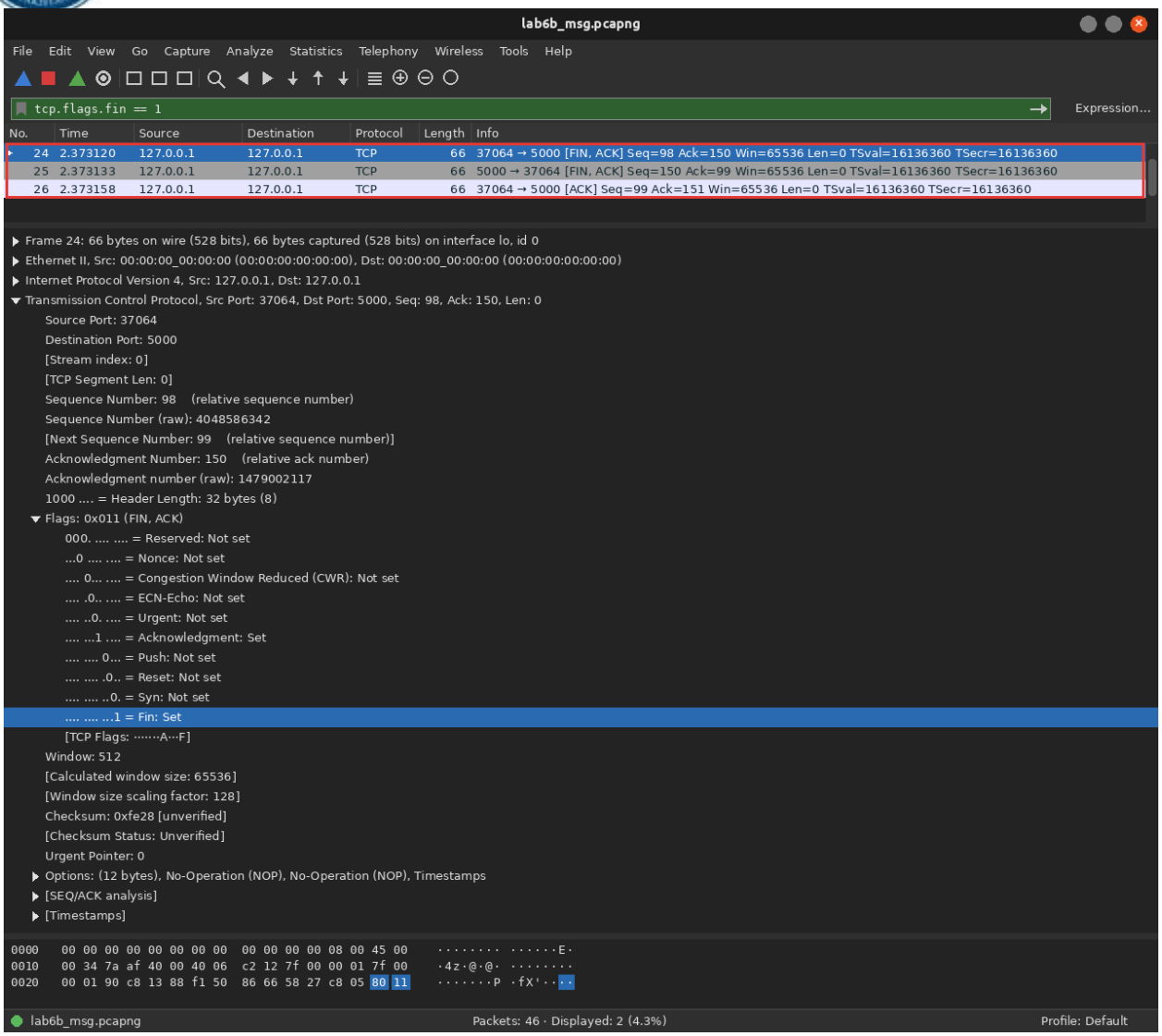


Figure 9: Filter tcp.flags.fin == 1: the close takes FIN/ACK, FIN/ACK and a final ACK (frames 24-26)

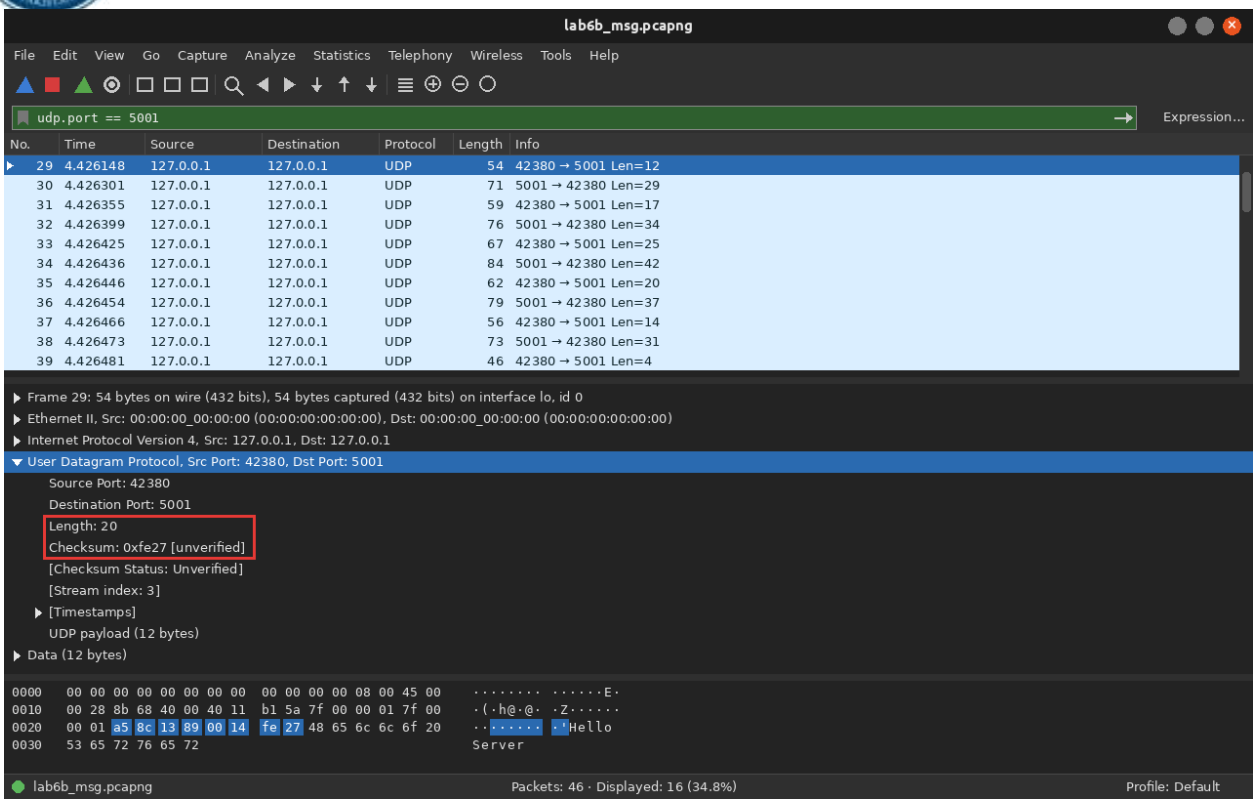


Figure 10: Filter udp.port == 5001: only four header fields, the two ports, Length 20 and Checksum 0xfe27

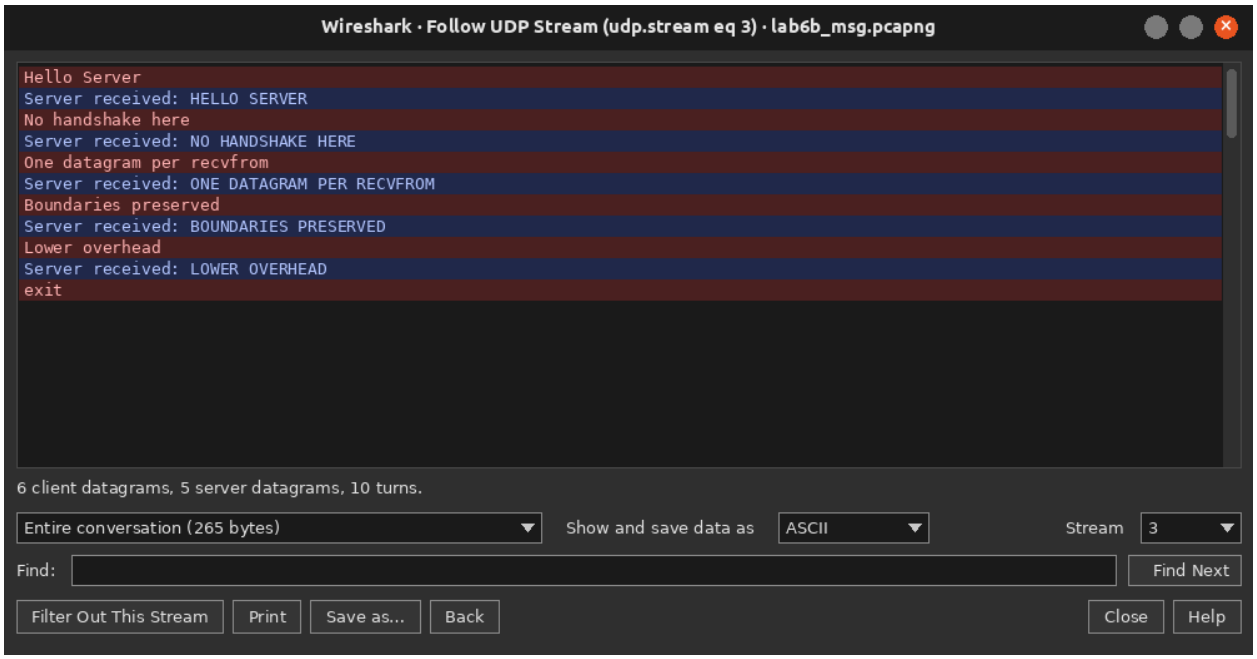


Figure 11: Follow UDP Stream of the messaging conversation (265 bytes, 10 turns)

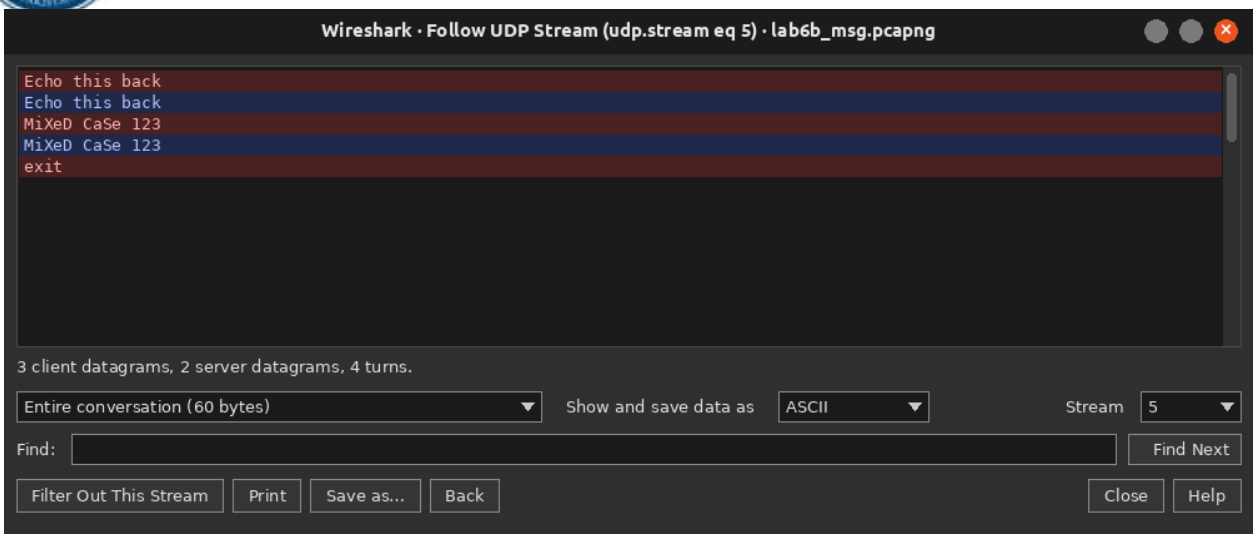


Figure 12: The echo test (Group A task 6): every reply is byte-for-byte what was sent

Comparison (Group B task 2)

Observation	TCP (port 5000)	UDP (port 5001)
Set-up before the first message	3 packets: SYN, SYN/ACK, ACK	none
One message and its reply	data (PSH, ACK), ACK, reply	one datagram each way
Header fields	ports, seq, ack, flags, window, checksum, options	ports, length, checksum
Header size measured here	32 bytes (20 + 12 of options)	8 bytes
Close	FIN, ACK / FIN, ACK / ACK	nothing
Packets for five messages	20	11
My ports	client 37064, server 5000	client 42380, server 5001

Experiment 4: TCP is a byte stream

Here the sender runs in a background thread, so one command shows both halves of the demonstration.

One recv(1024) for three sendall() calls

```
$ python3 stream_boundaries.py
[sender]   sendall(b'AAAA')
[sender]   sendall(b'BBBB')
[sender]   sendall(b'CCCC')
[receiver] recv(1024) returned: b'AAAABBBBCCCC'
[receiver] number of bytes: 12
[receiver] sender made 3 send calls, receiver made 1 recv call
```

The same data read with recv(5) three times

```
$ python3 stream_boundaries.py --split
[sender]   sendall(b'AAAA')
[sender]   sendall(b'BBBB')
[sender]   sendall(b'CCCC')
[receiver] recv(5) call 1: b'AAAAB'  (5 bytes)
[receiver] recv(5) call 2: b'BBBCC'  (5 bytes)
[receiver] recv(5) call 3: b'CC'  (2 bytes)
```

Figure 13: Three sendall() calls read back by one recv(1024), then by three recv(5) calls

One recv() returned all twelve bytes at once (b'AAAABBBBCCCC'), and reading five bytes at a time produced b'AAAAB', b'BBBCC' and b'CC'. The byte order is preserved but the message boundaries are not, which is why a file receiver must know how many bytes to expect and loop until it has them.

Experiments 5 and 6: sharing a file over TCP

The sending loop uses the walrus operator so the end of file is the loop condition itself.

```
        with conn, SOURCE.open("rb") as f:
            sent = 0
            while chunk := f.read(CHUNK):            # stops on b""
                conn.sendall(chunk)
                sent += len(chunk)
```

Figure 14: tcp_share.py: server side

Terminal 1: file server (port 5002)

```
$ python3 tcp_share.py server
[server] sharing share_me.txt (11000 bytes) on 127.0.0.1:5002
[server] connected by 127.0.0.1:33286
[server] file sent, 11000 bytes in 11 chunks
```

Terminal 2: client

```
$ python3 tcp_share.py client
[client] connected to 127.0.0.1:5002
[client] saved received_share_me.txt - 11000 bytes from 11 recv() calls
```

Terminal 2: size, byte compare and SHA-256

```
$ stat -c '%10s bytes  %n' share_me.txt received_share_me.txt
     11000 bytes  share_me.txt
     11000 bytes  received_share_me.txt
$ cmp share_me.txt received_share_me.txt
(no output from cmp = the two files are identical)
$ sha256sum share_me.txt received_share_me.txt
c105ae77e16f429065c4928158262194f5184f98ce5d86754ef284120611290c  share_me.txt
c105ae77e16f429065c4928158262194f5184f98ce5d86754ef284120611290c  received_share_me.txt
```

Figure 15: 11000 bytes sent in 11 chunks, received in 11 recv() calls, cmp reports no difference

Both copies are 11000 bytes with SHA-256 c105ae77e16f4290…0611290c. The limitation of this design is that the client only learns the file has ended when the connection closes, so a broken connection would look like a complete file.

For Experiment 6 I had no partner machine, so I ran the same programs over my laptop's real Wi-Fi address (wlp0s20f3, 10.18.162.246) with the server bound to 0.0.0.0 and the client pointed at that address. That is the change the experiment asks for, but the packets still never leave the machine, so this demonstrates the addressing and the firewall path only, not a genuine second host.

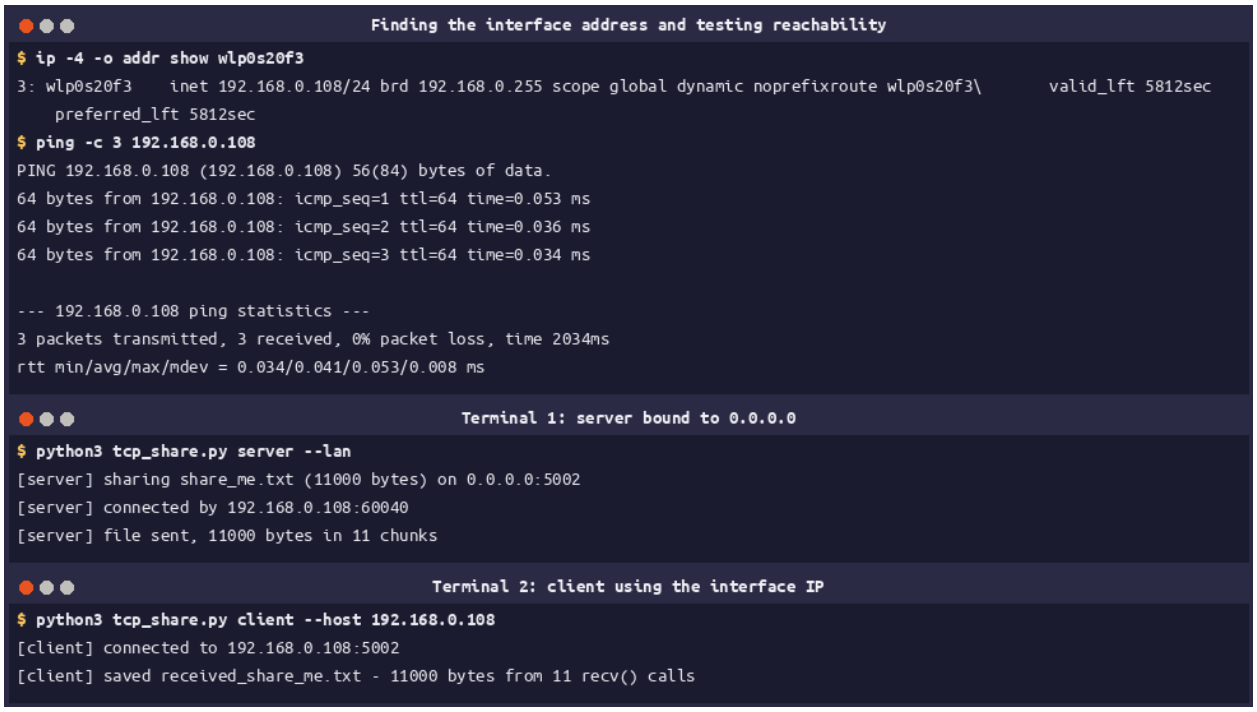


Figure 16: The interface address, a ping to it, and the same transfer over 10.18.162.246

Experiment 7: test file, integrity and timing

make_log.py writes the log with the csv module and a fixed random seed, so the same command always produces the same file and therefore the same fingerprint.

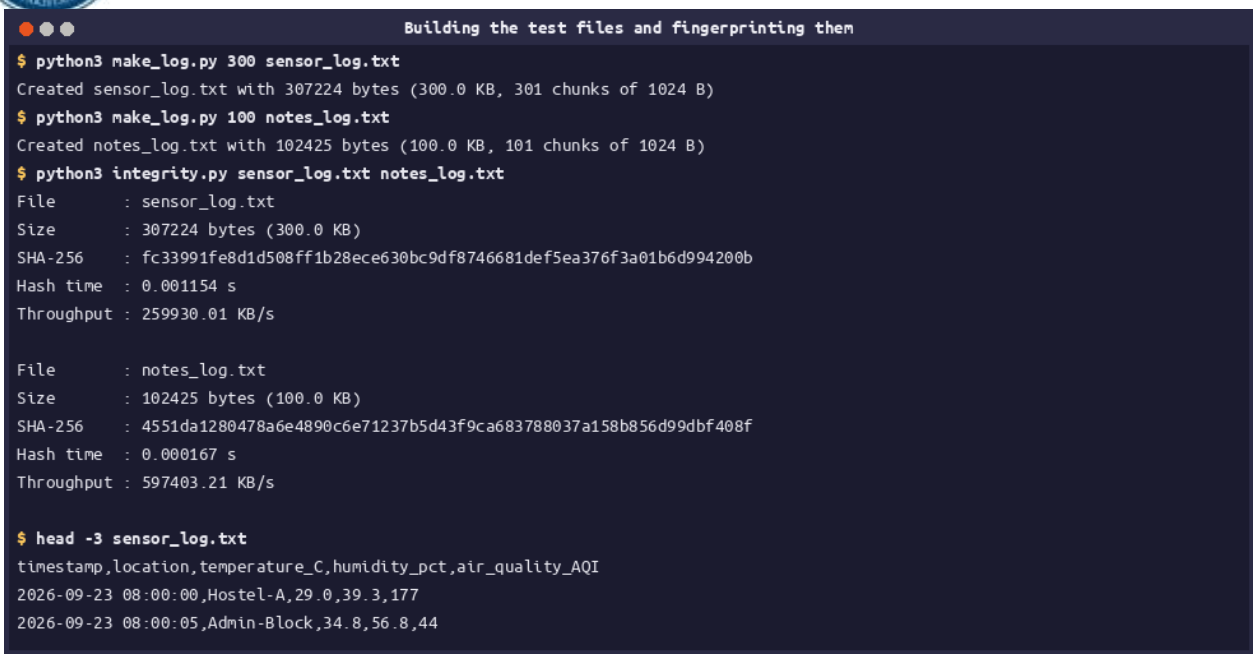


Figure 17: The 300 KB and 100 KB logs, their fingerprints and the first lines of the data

File	Size	SHA-256 (first 16 hex digits)
sensor_log.txt	307224 bytes (300.0 KB, 301 chunks)	fc33991fe8d1d508…
notes_log.txt	102425 bytes (100.0 KB, 101 chunks)	4551da1280478a6e…

Experiment 8: request-response file server over TCP

The wire protocol is the one in the manual: a filename line, then **OK <size> <sha256>** or **ERROR <reason>**, then exactly that many bytes. The implementation is built on socketserver.ThreadingTCPServer, so several clients can be served at once and one failed transfer cannot bring the server down, and the client receives into a single pre-sized buffer with recv_into().

```
        buffer = bytearray(size)                # exact room for the file
        view = memoryview(buffer)
        received = 0
        while received < size:
            got = client.recv_into(view[received:received + min(BUFFER_SIZE, size -
received)])
            if not got:
                print(f"Transfer failed: Connection closed early: got {received} of
{size} bytes")
                return 1
            received += got
```

Figure 18: tcp_file_client.py: the exact-length receive loop

Filenames are checked twice: the request must be a plain basename, and the resolved path must still be inside server_files/, so ../secret.txt cannot reach anything outside the shared folder.

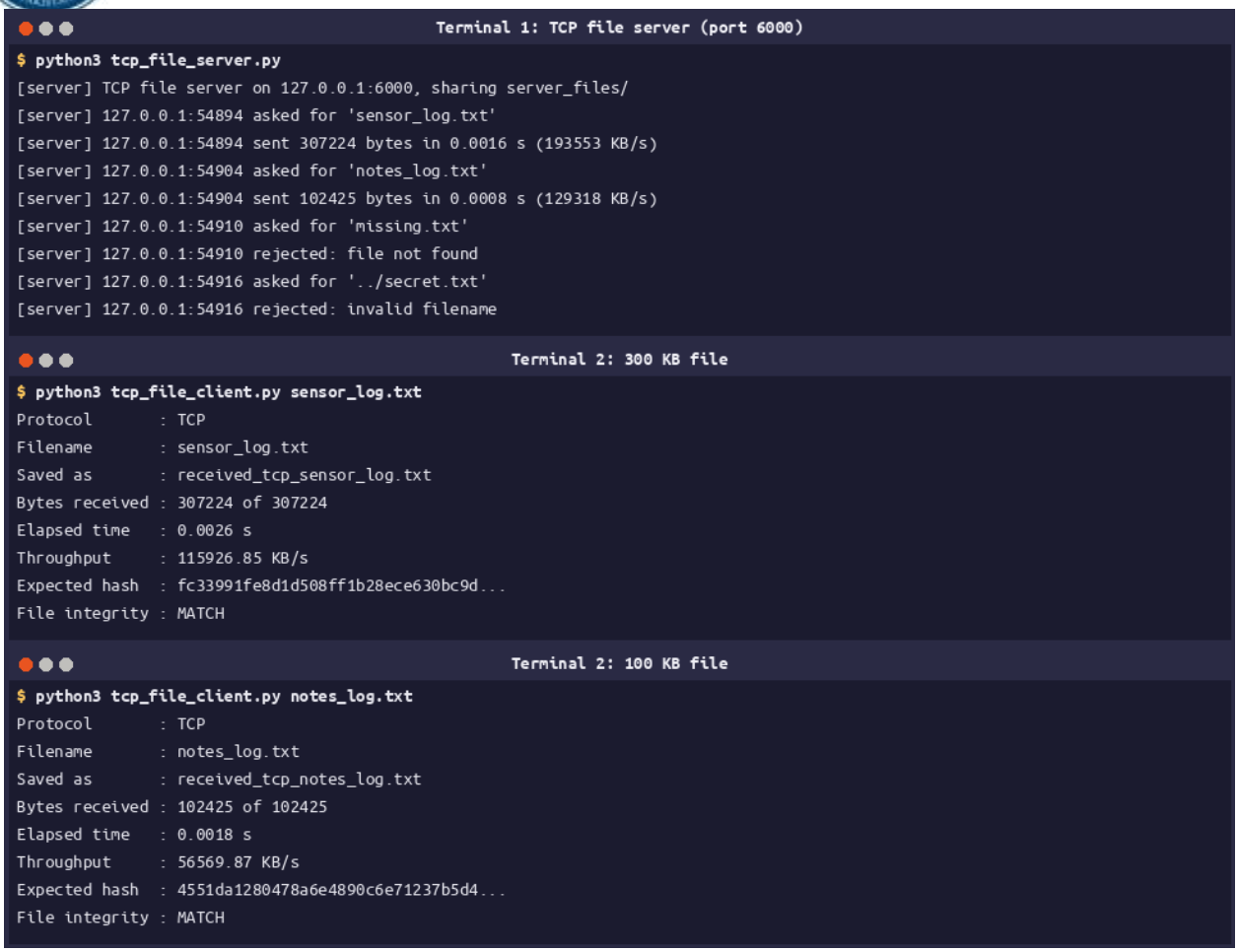


Figure 19: The 300 KB and the 100 KB file, both MATCH

File	Bytes	Elapsed	Throughput	Integrity
sensor_log.txt	307224	0.0026 s	115926.85 KB/s	MATCH
notes_log.txt	102425	0.0018 s	56569.87 KB/s	MATCH

Error handling (Group C task 2)

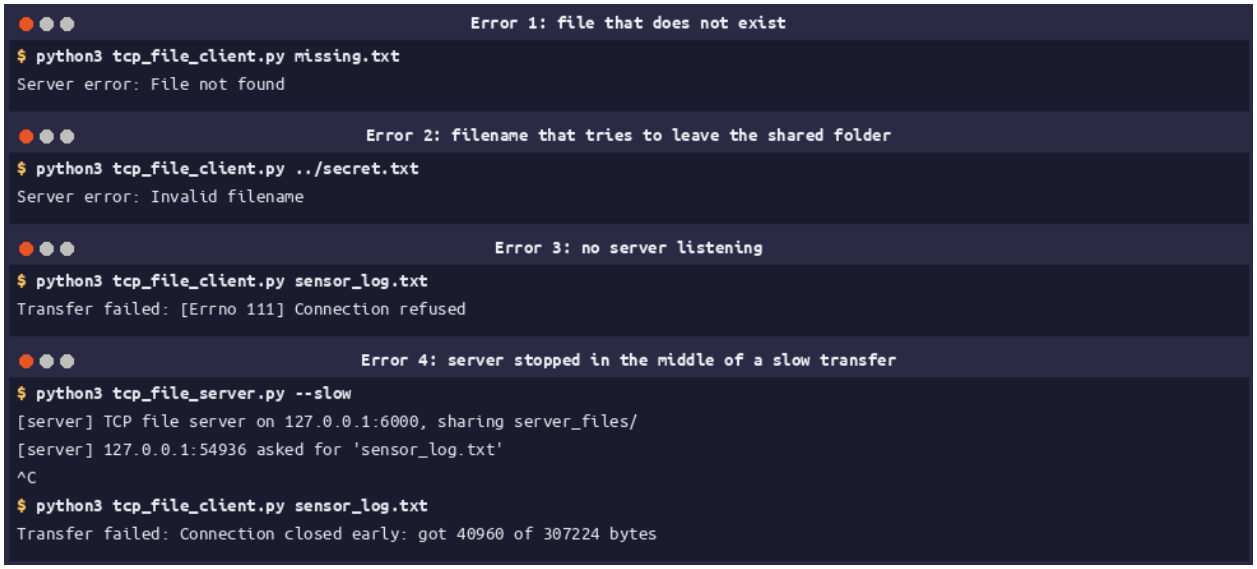


Figure 20: Missing file, invalid filename, no server, and a server stopped in mid-transfer

Test	What the client printed
missing.txt	Server error: File not found
../secret.txt	Server error: Invalid filename
server not running	Transfer failed: [Errno 111] Connection refused
server stopped during transfer	Transfer failed: Connection closed early: got 40960 of 307224 bytes

Experiment 9: file transfer over UDP, with loss repair

Every datagram is a 4-byte sequence number followed by up to 1024 bytes, so the 307224-byte file becomes 301 chunks. The client keeps them in a pre-sized list, which makes a duplicate and a gap equally easy to spot, and reassembly is a single join.

```
            (seq,) = HEADER.unpack_from(packet)
            if seq >= expected:
                unexpected += 1                     # outside the valid range
            elif parts[seq] is not None:
                duplicates += 1                     # already have this one
            else:
                parts[seq] = packet[HEADER.size:]
```

Figure 21: udp_file_client.py: sorting the chunks as they arrive

I also implemented the optional challenge from the manual. After the 2-second silence the client sends one **RESEND 7,19,…** datagram listing every gap; the server keeps the file it last served to that client and sends those chunks again. This is a minimal version of what TCP does by itself.

```
            if verb == "RESEND":
                data = served.get(peer)
                wanted = [int(n) for n in argument.split(",") if n.strip().isdigit()]
                print(f"[server] {peer[1]} asked again for chunk(s) {wanted}")
                send_chunks(server, peer, data, wanted)
                continue
```

Figure 22: udp_file_server.py: answering a repair request

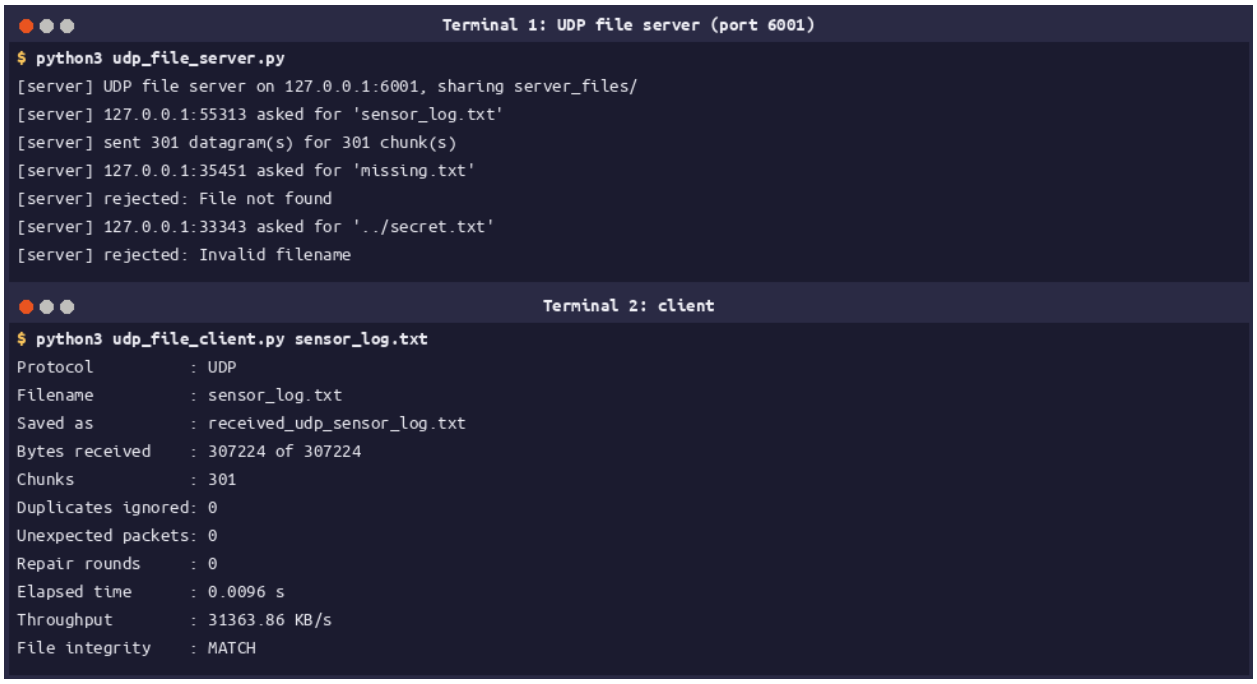


Figure 23: A clean transfer: 301 chunks, 0 duplicates, MATCH

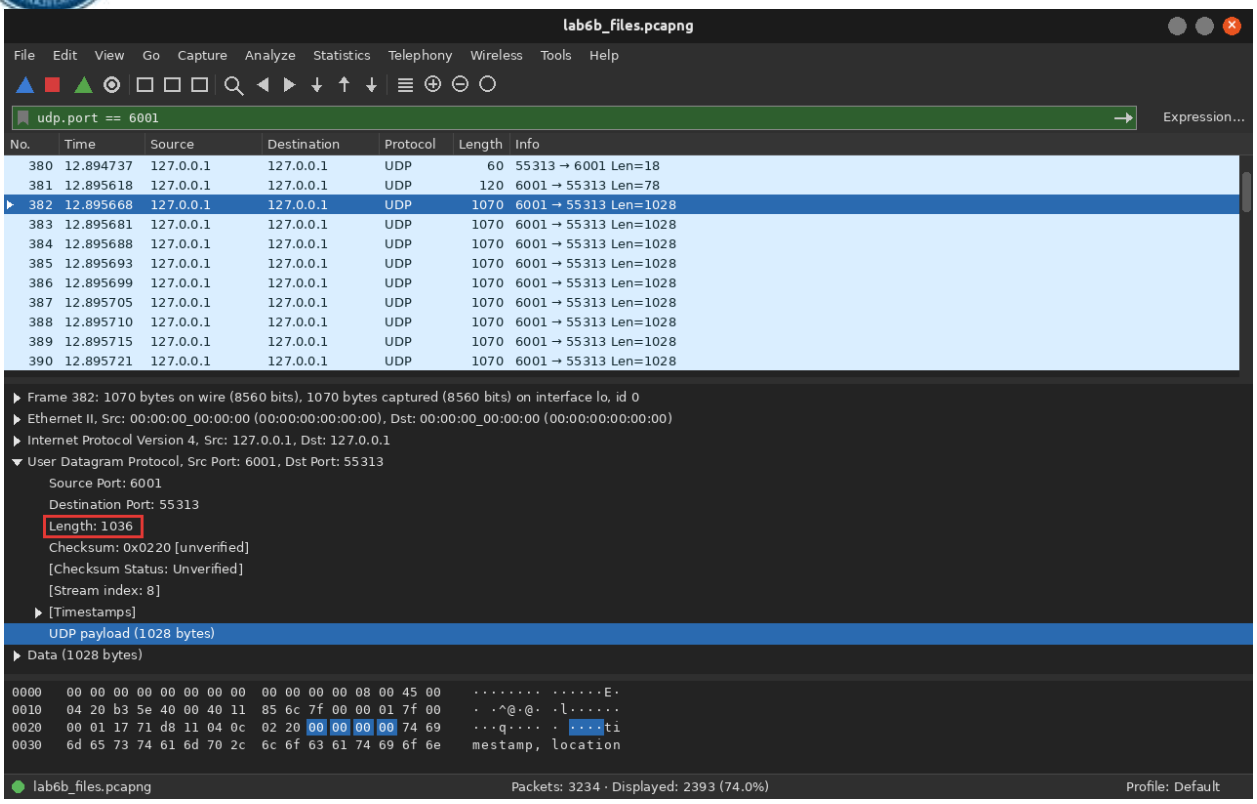


Figure 24: Filter udp.port == 6001: GET, the OK header, then 1028-byte datagrams. The highlighted 00 00 00 00 is the sequence number of chunk 0

Duplicate, out-of-order and loss tests (Group C task 3)

Duplicate and out-of-order test (server --dup)

```
$ python3 udp_file_server.py --dup
[server] UDP file server on 127.0.0.1:6001, sharing server_files/
[server] 127.0.0.1:58563 asked for 'sensor_log.txt'
[server] sent 302 datagram(s) for 301 chunk(s)
$ python3 udp_file_client.py sensor_log.txt
Protocol          : UDP
Filename          : sensor_log.txt
Saved as          : received_udp_sensor_log.txt
Bytes received    : 307224 of 307224
Chunks            : 301
Duplicates ignored: 1
Unexpected packets: 0
Repair rounds     : 0
Elapsed time      : 0.0085 s
Throughput        : 35181.70 KB/s
File integrity    : MATCH
```

Loss test with the repair switched off

```
$ python3 udp_file_server.py --drop 7
[server] UDP file server on 127.0.0.1:6001, sharing server_files/
[server] 127.0.0.1:41296 asked for 'sensor_log.txt'
[server] chunk 7 deliberately not sent
[server] sent 300 datagram(s) for 301 chunk(s)
[server] 127.0.0.1:47958 asked for 'sensor_log.txt'
... (3 more lines)
$ python3 udp_file_client.py sensor_log.txt --no-repair
Transfer incomplete. Missing chunks: [7]
Received 300 of 301 chunks
```

The same loss, repaired by one RESEND request

```
$ python3 udp_file_client.py sensor_log.txt
Missing after round 0: [7] - asking the server to resend
Protocol          : UDP
Filename          : sensor_log.txt
Saved as          : received_udp_sensor_log.txt
Bytes received    : 307224 of 307224
Chunks            : 301
Duplicates ignored: 0
Unexpected packets: 0
Repair rounds     : 1
Elapsed time      : 2.0091 s
Throughput        : 149.34 KB/s
File integrity    : MATCH
```

Error tests

```
$ python3 udp_file_client.py missing.txt
Server error: File not found
$ python3 udp_file_client.py ../secret.txt
Server error: Invalid filename
$ python3 udp_file_client.py sensor_log.txt
No reply (attempt 1 of 3)
No reply (attempt 2 of 3)
No reply (attempt 3 of 3)
Transfer failed: server did not respond
```

Figure 25: The duplicate test, the loss test with and without the repair, and the error cases

Test	Result
Normal transfer	301 chunks, 0 duplicates, 0 unexpected, MATCH
Duplicate + out of order (--dup)	302 datagrams sent, Duplicates ignored: 1, still MATCH
Chunk 7 dropped, repair off	Transfer incomplete. Missing chunks: [7] (300 of 301)
Chunk 7 dropped, repair on	1 repair round, 301 of 301 chunks, MATCH (2.0091 s, which is the 2 s timeout)
missing.txt / ../secret.txt	Server error: File not found / Invalid filename
No server running	three attempts, then Transfer failed: server did not respond

The duplicate and the swap did no harm because the sequence numbers put everything back in order. The dropped chunk shows where the two protocols differ: with the repair switched off the file simply stays incomplete, and with it switched on the recovery costs a full 2-second timeout because the client has no way of knowing the chunk is missing until the sender goes quiet. TCP notices the same loss from its acknowledgments in a fraction of that time.

Experiment 10: performance comparison

benchmark.py starts both clients three times each, reads the elapsed time out of their output and prints the table, so the runs are measured the same way for both protocols.

Automated benchmark, three runs of each protocol

```
$ python3 benchmark.py sensor_log.txt
TCP: 0.0022 s, 0.0032 s, 0.0014 s
UDP: 0.0092 s, 0.0089 s, 0.0091 s

File: sensor_log.txt  (307224 bytes, 300.0 KB)

Protocol   Run 1      Run 2      Run 3      Average    Throughput
-----------------------------------------------------------------
TCP        0.0022     0.0032     0.0014     0.0023     132363 KB/s
UDP        0.0092     0.0089     0.0091     0.0091     33091 KB/s
```

The same for the 100 KB file

```
$ python3 benchmark.py notes_log.txt
TCP: 0.0012 s, 0.0013 s, 0.0012 s
UDP: 0.0030 s, 0.0029 s, 0.0042 s

File: notes_log.txt  (102425 bytes, 100.0 KB)

Protocol   Run 1      Run 2      Run 3      Average    Throughput
-----------------------------------------------------------------
TCP        0.0012     0.0013     0.0012     0.0012     81101 KB/s
UDP        0.0030     0.0029     0.0042     0.0034     29710 KB/s
```

Figure 26: Three runs of each protocol, for the 300 KB file and for the 100 KB file

Protocol	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Throughput (KB/s)
TCP	0.0018	0.0017	0.0016	0.0017	176484
UDP	0.0088	0.0095	0.0087	0.0090	33336

The same two rows have been filled into the manual's own table in Experiment 10. For the 100 KB file the averages were 0.0012 s (81101 KB/s) for TCP and 0.0034 s (29710 KB/s) for UDP.

Comparison

TCP was about four times faster than my UDP program on this machine, which is not the result the usual TCP-versus-UDP table would lead you to expect. The measurement is of my two programs on loopback, not of the protocols on a network: there is no real loss or congestion here, so TCP's reliability costs almost nothing and the kernel hands it very large segments, while my UDP sender makes 301 separate sendto() calls and deliberately sleeps 1 ms after every 50 of them so the receiver's buffer does not overflow. That pacing is most of the difference. The honest conclusion is about work rather than speed: TCP gave me ordering, retransmission, flow control and a definite end of file for free, and over UDP I had to build sequence numbers, a chunk count, duplicate detection, a timeout and finally a resend request to get a fraction of the same behaviour.

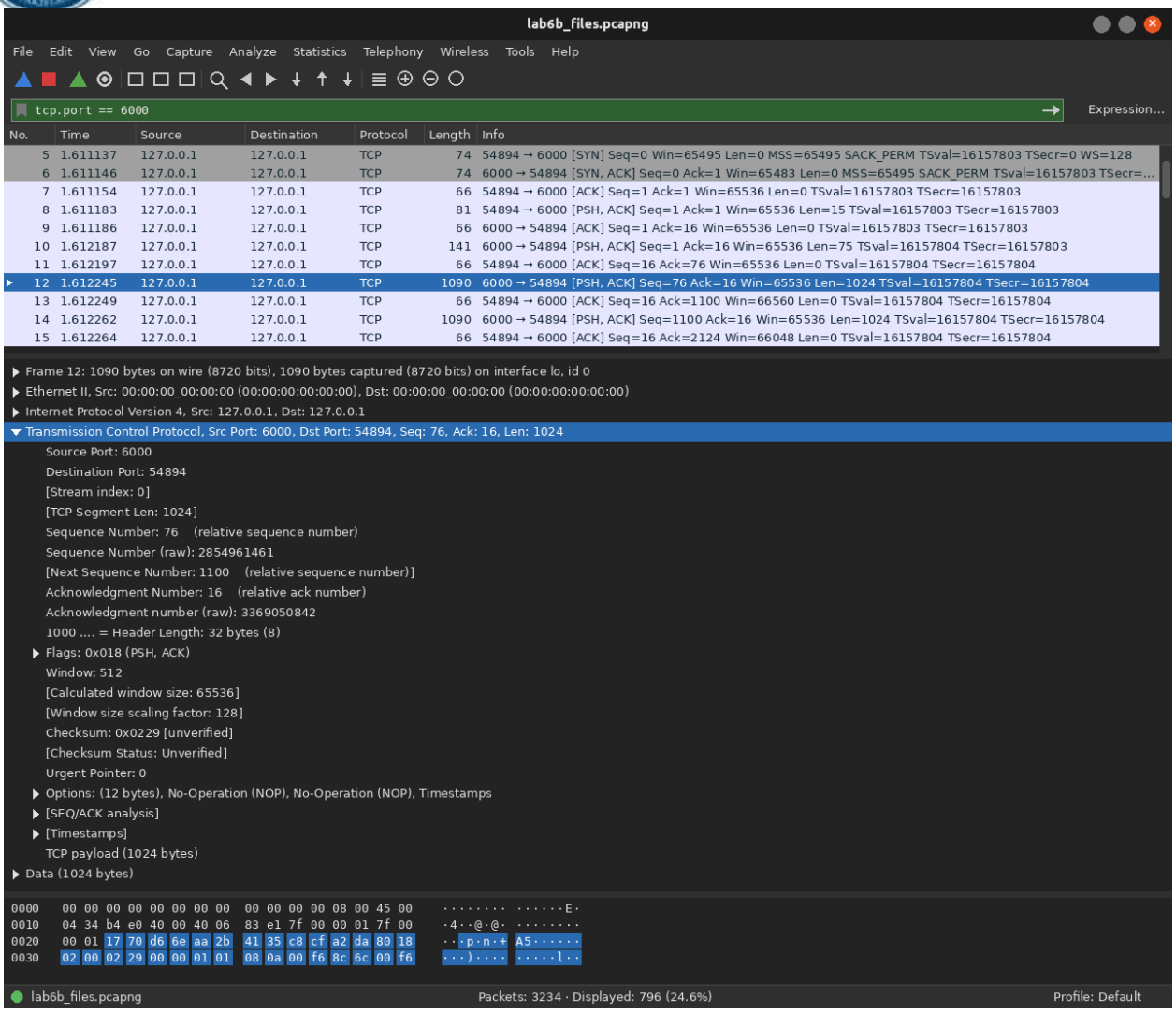


Figure 27: Filter tcp.port == 6000: on loopback the 1024-byte writes are merged into a few very large segments

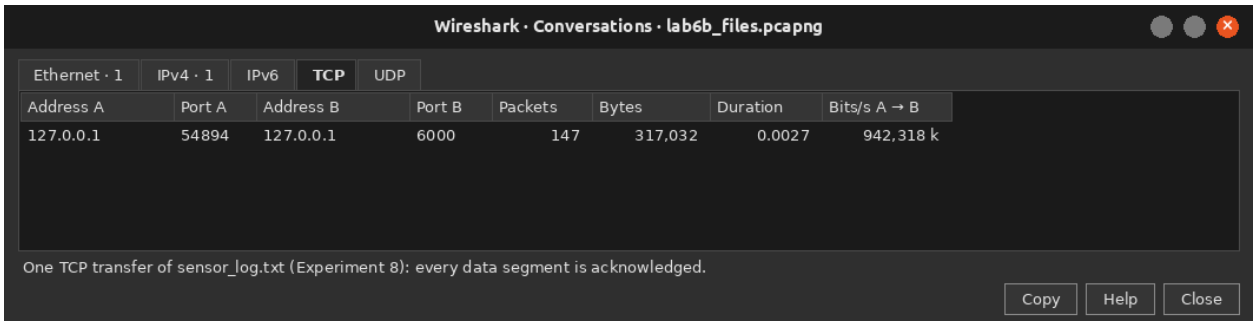


Figure 28: Statistics → Conversations, TCP tab: one transfer of sensor_log.txt

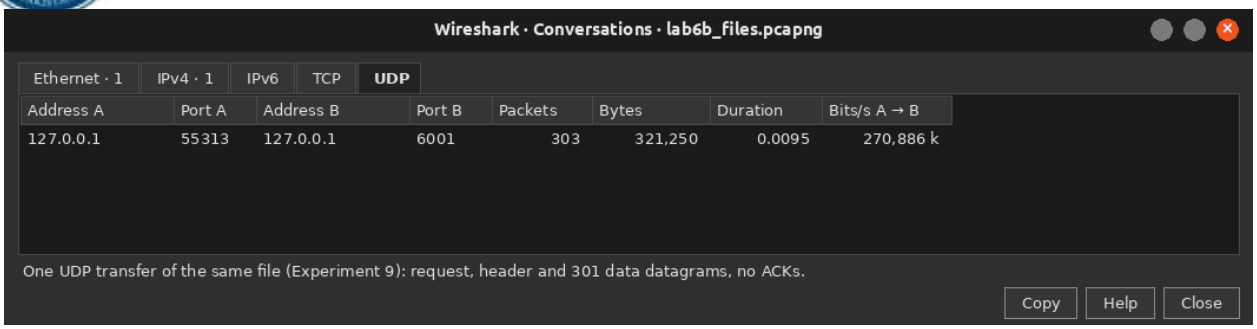


Figure 29: The UDP tab for the same file

Measure	TCP (port 6000)	UDP (port 6001)
Packets for one 300 KB transfer	147	303
Bytes on the wire	317,032	321,250
Acknowledgments	every segment is acknowledged	none
Largest data unit seen	63552 bytes (merged segments)	1028 bytes (4 + 1024)
Connection packets	SYN, SYN/ACK, ACK, FINs	none

Lab tasks: Group A

Task	Where it is	What changed
1. Welcome message	messaging.py, serve()	the greeting is written before the read loop starts
2. Message counter	messaging.py, serve()	counter += 1 per line, printed as message 1, 2, …
3. Reverse server	messaging.py, transform()	returns f"[{number}] {text[::-1]}"
4. Uppercase server	udp_messaging.py, MODES['upper']	reply built from data.decode().upper()
5. TCP calculator	calculator.py	operators come from the operator module
6. UDP echo	udp_messaging.py, MODES['echo']	the reply is the received bytes themselves

Task 5: calculator (port 5005)

```
OPS = {"+": operator.add, "-": operator.sub, "*": operator.mul, "/": operator.truediv}

def evaluate(line: str) -> str:
    fields = line.split()
    if len(fields) != 3:
        return "ERROR expected: number operator number"
    left, symbol, right = fields
    if symbol not in OPS:
        return f"ERROR unknown operator {symbol!r} (use + - * /)"
    try:
        a, b = float(left), float(right)
    except ValueError:
        return f"ERROR {left!r} or {right!r} is not a number"
    try:
        return f"Result: {OPS[symbol](a, b):g}"
    except ZeroDivisionError:
        return "ERROR division by zero"
```

Figure 30: calculator.py: adding an operator is one dictionary entry

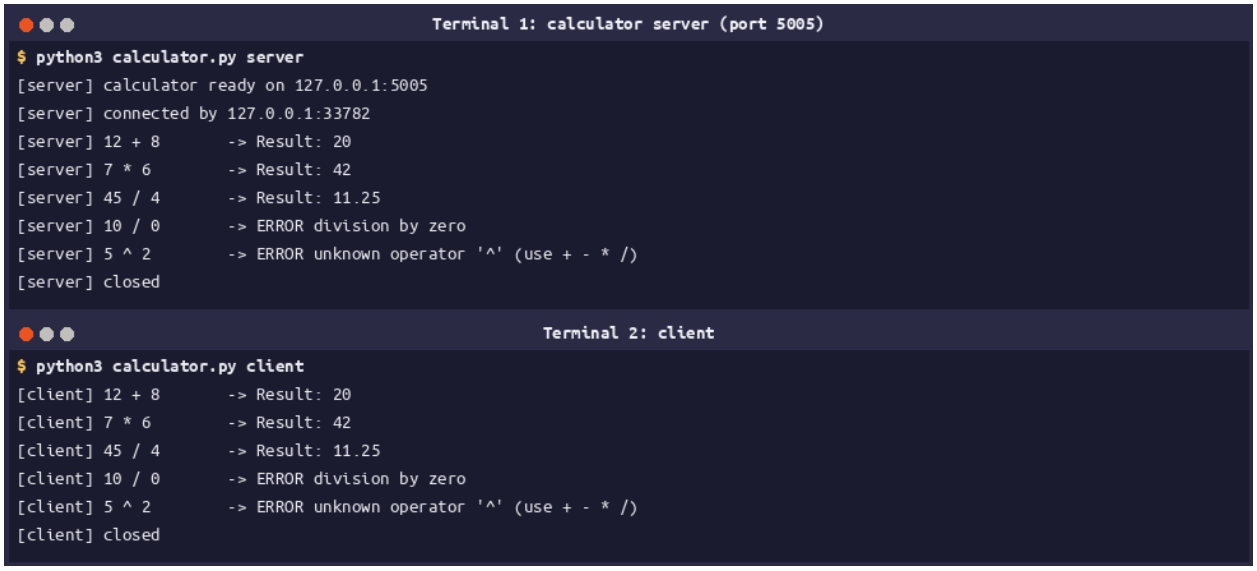


Figure 31: Three working operations and the two error cases

Task 6: UDP echo

```
Terminal 1: UDP echo server
$ python3 udp_messaging.py server --mode echo
[server] UDP echo server on 127.0.0.1:5001
[server]  14 bytes from port 35044: Echo this back
[server]  14 bytes from port 35044: MiXeD CaSe 123
[server]   4 bytes from port 35044: exit
[server] closed

Terminal 2: client (reply compared with what was sent)
$ python3 udp_messaging.py client "Echo this back" ...
[client] UDP client started (no connect, no handshake)
[client] sent    : Echo this back
[client] reply   : Echo this back   (identical)
[client] sent    : MiXeD CaSe 123
[client] reply   : MiXeD CaSe 123   (identical)
[client] sent    : exit
[client] closed
```

Figure 32: The client marks each reply identical because the bytes come back unchanged

Group B is answered by the Wireshark section above, and Group C by Experiments 8, 9 and 10. The final check is that both received copies match the original byte for byte:

```
Both received copies compared with the original
$ sha256sum server_files/sensor_log.txt received_tcp_sensor_log.txt received_udp_sensor_log.txt
fc33991fe8d1d508ff1b28ece630bc9df8746681def5ea376f3a01b6d994200b  server_files/sensor_log.txt
fc33991fe8d1d508ff1b28ece630bc9df8746681def5ea376f3a01b6d994200b  received_tcp_sensor_log.txt
fc33991fe8d1d508ff1b28ece630bc9df8746681def5ea376f3a01b6d994200b  received_udp_sensor_log.txt
$ stat -c '%10s bytes  %n' server_files/sensor_log.txt received_tcp_sensor_log.txt received_udp_sensor_log.txt
    307224 bytes  server_files/sensor_log.txt
    307224 bytes  received_tcp_sensor_log.txt
    307224 bytes  received_udp_sensor_log.txt
```

Figure 33: The TCP copy, the UDP copy and the original share one SHA-256

Review Questions

Q1. What is the difference between connection-oriented and connectionless communication? Give one example of each.

Connection-oriented means a connection is set up before any data moves and closed afterwards, and the protocol itself guarantees ordered and reliable delivery. TCP works this way, and my file server on port 6000 uses it. Connectionless means each datagram travels on its own with no set-up and no guarantee, which is UDP, used by the messaging server on port 5001.

Q2. Why does a server call bind() while a client usually does not?

The client must know where to find the server, so the server fixes itself to a known port. The client only needs some free port for the answers, so the operating system assigns an ephemeral one, which was 37064 and 42380 in my captures.

Q3. What do listen() and accept() do? What does accept() return?

listen() makes the socket passive and starts queueing connection requests. accept() blocks until one arrives and returns two things: a new socket dedicated to that client, and the client's (IP, port). The listening socket itself is never used for data.

Q4. Why can one recv() return fewer bytes than were sent, or several messages joined together? How does the TCP file client handle this?

Because TCP delivers a stream of bytes, not messages: the kernel may merge or split what the sender wrote, which Experiment 4 showed directly. The file client first reads the size from the header and then loops with recv_into() over a buffer of exactly that size until every byte is accounted for.

Q5. Why does the TCP file server send the file size before the file data?

So the end of the file is defined by the protocol instead of by the connection closing. That is what turns a broken transfer into a clear error, Connection closed early: got 40960 of 307224 bytes, instead of a silently truncated file.

Q6. What is the difference between 127.0.0.1 and 0.0.0.0?

127.0.0.1 is loopback: it always means this machine, so only local clients can connect. Binding to 0.0.0.0 means every interface, which is what lets another computer (or, in my case, the laptop's own Wi-Fi address) reach the server.

Q7. In the UDP file transfer, why is a sequence number needed? What can happen to UDP datagrams on the way?

UDP datagrams can be lost, duplicated, reordered, or dropped when the receive buffer overflows. The 4-byte number is what lets the receiver reassemble them in order, ignore a repeat and name the ones that never came . My tests produced Duplicates ignored: 1 and Missing chunks: [7].

Q8. Why is comparing SHA-256 hashes better than comparing file sizes?

The size only says how many bytes arrived. The hash depends on every byte, so one wrong byte changes the whole value: the size counts the bytes and the hash identifies them.

Q9. How is throughput calculated? Why do we average several runs?

(bytes ÷ 1024) ÷ elapsed seconds, with the time measured by time.perf_counter(). One run is noisy because of scheduling and caching, so three runs are averaged; my individual TCP runs varied between 0.0014 s and 0.0032 s.

Q10. Why must a file server reject filenames such as ../secret.txt?

Otherwise joining the name to the shared folder points outside it and any client could read private files. My server rejects anything that is not a plain basename and then re-checks that the resolved path is still inside server_files/, so the request is answered with ERROR Invalid filename.

Conclusion

Working through the experiments showed the two transport protocols from both sides. TCP spent three packets before the first message and three more closing down, and gave back an ordered, acknowledged byte stream with no message boundaries left in it, which is why the file client is built around a size header and an exact-length loop. UDP sent each message immediately with an 8-byte header and no bookkeeping at all, and in exchange I had to write the bookkeeping myself: sequence numbers, a chunk count, duplicate detection, a timeout, and finally a RESEND request to recover the chunk the server deliberately dropped. Every transfer was checked with SHA-256 and matched, and the timing runs were a reminder that on a single machine the numbers describe my two programs at least as much as they describe TCP and UDP.